



دانشکده فنی مهندسی - گروه کامپیوتر

پایان نامه کارشناسی

بازی تحت وب Fight CoronaVirus

دانشجو : محیا امیری (۹۶۱۱۱۳۰۰۴)

استاد راهنما : دکتر سید امیرهادی مینوفام

تیر ۱۴۰۰

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

دانشگاه علم و فرهنگ

دانشکده فنی مهندسی

گروه کامپیوتر

پایان نامه کارشناسی

بازی تحت وب Fight CoronaVirus

دانشجو : محیا امیری

استاد راهنما : دکتر سید امیرهادی مینوفام

چکیده

با توجه به بیماری کرونا که این روزها تمام جهان را فرا گرفته است، ساخت بازی‌های کامپیوتری مرتبط با این بیماری، می‌تواند به کودکان در زمینه‌ی درک اهمیت بیماری و رعایت پروتکل‌های بهداشتی کمک کند. امروزه زندگی کودکان از سنین پایین، با فضای مجازی عجین شده است. بنابراین خوب است محتواهایی را در اختیار آن‌ها قرار دهیم که مفید باشد.

طی این دو سال اخیر که کرونا شیوع یافته، بازی‌های مرتبط زیادی، با درجه سختی و امکانات مختلف (اعم از تک نفره/ چند نفره بودن بازی و ...) ساخته شده‌اند.

بازی‌ای که طراحی کرده‌ام، بازی تخیلی و ساده می‌باشد و برای سنین مختلف هم مناسب است. این بازی جنبه‌ی آموزشی دارد و به کاربر نشان می‌دهد که برای جلوگیری از مبتلا شدن به این ویروس، وجود الکل لازم است یا مثلاً اینکه در صورت برخورد با ویروس، انرژی به طور مداوم کم می‌شود.

کلمات کلیدی: کرونا، بازی، پروتکل‌های بهداشتی، فضای مجازی

تقدیم به مادر بزرگ عزیزم

که همواره در زندگی حامی و مشوق من است.

سپاس و قدردانی ویژه از

اساتید گرامی

دکتر سید امیرهادی مینوفام

محمدامیر فرشچی

فراز سامعی

و

دکتر یوسف مهرداد

که در این راه، دلسوزانه مرا یاری نمودند.

فهرست

فصل اول - مقدمه و کلیات پروژه	۱
۱-۱. مقدمه	۲
۱-۲. ویژگی‌های Three.js	۴
۱-۲-۱. مزایا	۴
۱-۲-۲. معایب	۴
۱-۳. معرفی پروژه	۶
فصل دوم - معرفی نرم افزار	۷
۲-۱. Visual Studio Code	۸
۲-۱-۱. کاربردها و ویژگی‌ها	۸
۲-۱-۲. قابلیت‌ها و مزایا	۹
فصل سوم - نحوه‌ی کار با نرم افزار	۱۱
۳-۱. دانلود نرم افزار	۱۲
۳-۲. نصب	۱۲
۳-۳. ایجاد فایل	۱۷

فهرست

۳-۴. نمایش خروجی	۱۸
فصل چهارم - طراحی	۲۰
۴-۱. کتابخانه و ساختار کلی	۲۱
۴-۲. درخت DOM	۲۲
۴-۳. تابع animate()	۲۴
۴-۴. ساخت صحنه‌ی بازی	۲۵
۴-۴-۱. صحنه	۲۵
۴-۴-۲. دوربین	۲۵
۴-۴-۲-۱. دوربین آرایه‌ای	۲۷
۴-۴-۲-۲. دوربین مکعبی	۲۷
۴-۴-۲-۳. دوربین عمودی	۲۷
۴-۴-۲-۴. دوربین پرسپکتیو	۲۸
۴-۴-۲-۵. دوربین استریو	۲۸
۴-۴-۳. Render	۲۸

فهرست

فصل پنجم - Front-end پروژه ۳۰

۳۱ ۵-۱. صفحه‌ی شروع بازی

۳۳ ۵-۱-۱. دکمه Start

۳۳ ۵-۱-۲. Energy Bar

۳۳ ۵-۱-۳. امتیاز

۳۴ ۵-۲. صفحه‌ی پس‌زمینه‌ی بازی

۳۷ ۵-۳. صفحه‌ی باخت بازی

۳۸ ۵-۳-۱. دکمه Restart

فصل ششم - Back-end پروژه ۳۹

۴۰ ۶-۱. مدیریت انرژی

۴۰ ۶-۲. مدیریت امتیاز

۴۱ ۶-۳. سایه

۴۱ ۶-۴. سطح

۴۲ ۶-۵. بازیکن (آمبولانس)

فهرست

۴۲	۶-۶. موانع
۴۴	۶-۷. مکانیزم پرتاب الکل
۴۴	۶-۷-۱. Spray Gun
۴۵	۶-۷-۲. Bullet
۴۵	۶-۸. برخورد
۴۶	۶-۸-۱. برخورد آمبولانس با ویروس‌ها
۴۶	۶-۸-۲. برخورد الکل با ویروس‌ها
۴۷	۶-۹. نور
۴۸	۶-۱۰. صوت
۴۸	۶-۱۱. حرکت
۴۹	۶-۱۲. دور شدن آمبولانس از دوربین
۵۰	۶-۱۳. تابع bounce()
۵۱	فصل هفتم - نتیجه‌گیری
۵۲	۷-۱. نتیجه‌گیری

فهرست

منابع ۵۴

لیست منابع ۵۵

پیوست ۵۶

پیوست ۱- انواع سایه و ویژگی های آن ۵۷

پیوست ۲- انواع نور و ویژگی های آن ۵۹

فهرست اشکال

شکل ۱-۱		۵
شکل ۳-۱		۱۲
شکل ۳-۲		۱۳
شکل ۳-۳		۱۴
شکل ۳-۴		۱۵
شکل ۳-۵		۱۶
شکل ۳-۶		۱۷
شکل ۳-۷		۱۸
شکل ۳-۸		۱۹
شکل ۳-۹		۱۹
شکل ۴-۱		۲۲
شکل ۴-۲		۲۳
شکل ۵-۱		۳۰
شکل ۵-۲		۳۱

فهرست اشکال

۳۲	 شکل ۵-۳
۳۴	 شکل ۵-۴
۳۵	 شکل ۵-۵
۳۶	 شکل ۵-۶
۳۷	 شکل ۵-۷
۳۸	 شکل ۵-۸
۴۳	 شکل ۶-۱
۴۴	 شکل ۶-۲
۴۸	 شکل ۶-۳
۵۲	 شکل ۷-۱
۶۰	 شکل ۱.پ
۶۱	 شکل ۲.پ
۶۱	 شکل ۳.پ
۶۲	 شکل ۴.پ

فهرست اشکال

شکل پ.۵ ۶۳

شکل پ.۶ ۶۴

شکل پ.۷ ۶۵

شکل پ.۸ ۶۶

فصل اول

مقدمه و کلیات پروژه

تاریخچه‌ی بازی‌های کامپیوتری، به اوایل دهه‌ی ۵۰ میلادی بر می‌گردد، زمانی که دانشجویان رشته‌ی کامپیوتر برای کارهای پژوهشی خود، به ساخت و طراحی این بازی‌ها می‌پرداختند. اکنون با گذشت نزدیک به ۷۰ سال، شاهد همه گیر شدن بازی‌های کامپیوتری در سبک‌های گوناگون و روی پلتفرم‌های مختلف هستیم. یکی از این نوع‌ها، بازی‌های تحت وب هستند. با پیشرفت و گسترش اینترنت و مرورگرها، ساخت بازی‌های قابل اجرا روی مرورگرها، سرعت چشمگیری یافت. این بازی‌ها ژانرهای مختلفی داشتند و امکان بازی به صورت تک نفره، دو یا چند نفره هم برای افراد در نظر گرفته شده بود.

بازی‌های تحت وب برای کاربران به صورت رایگان در اینترنت موجود بوده و افراد می‌توانند با مرورگرهای مختلف آن‌ها را امتحان کنند. برای استفاده از این بازی‌ها، نیاز به دانلود و نصب فایل خاصی نیست و تنها کاری که باید انجام دهید، وارد نمودن آدرس وبسایت سرویس مورد نظر در مرورگرتان است.

در ابتدا بازی‌های تحت وب توسط HTML و تکنولوژی‌های HTML scripting مانند ASP, JavaScript, PHP و MySQL به صورت بسیار ابتدایی ساخته می‌شدند. با پیشرفت تکنولوژی‌های گرافیکی مبتنی بر وب مثل فلش و جاوا، این اجازه به سازندگان بازی‌ها داده شد تا بتوانند بازی‌های بهتری را طراحی کنند.

امروزه حتی با سایت‌هایی مواجه می‌شویم که گرافیک بسیار زیبایی دارند و بدون استفاده از فایل‌های Gif یا ویدئو، دارای اشکال سه بعدی و متحرک‌اند. این سایت‌ها برای انجام کارهای گرافیکی خود از WebGL استفاده می‌کنند. WebGL (مخفف Web Graphics Library)، یک کتابخانه‌ی JavaScript می‌باشد که با استفاده از آن می‌توان اشیاء تعاملی دو بعدی و سه بعدی ایجاد کرد. WebGL با تمام مرورگرها سازگار است و تکنولوژی‌ای cross-platform است که برای ارائه کردن تصاویر، مستقیماً از کارت گرافیک استفاده می‌کند.

بنابراین بهتر است برای دریافت تصاویر بهتر، همیشه درایور کارت گرافیک خود را به آخرین نسخه‌ی موجود روزرسانی کنید. WebGL به هیچ نرم افزار خاص یا پلاگینی وابسته نیست و فقط با استفاده از HTML5 و JavaScript می‌توان آن را پیاده‌سازی کرد. پیش از این گرافیک سه بعدی، محدود به کنسول‌های بازی و یا کامپیوترهای سطح بالا بود ولی امروزه با پیشرفت کامپیوترهای شخصی و مرورگرهای اینترنت، نمایش گرافیک سه بعدی از طریق تکنولوژی‌های مدرن و شناخته شده‌ی وب امکان پذیر است.

WebGL به صورت پیشفرض دارای امکانات کمی است. بنابراین بسیاری از موتورهای بازی‌سازی و کتابخانه‌های مختلف به وجود آمده‌اند که کارایی این مورد را بالاتر برده‌اند و استفاده‌پذیری آن را بیشتر ساخته‌اند. علاوه بر این، برای انجام هر کار مفیدی در WebGL، به تعداد خط زیادی کد احتیاج است. به همین دلیل کتابخانه‌ای به نام Three.js ایجاد شد که کار برنامه نویس را راحت‌تر کرد.

Three.js کتابخانه‌ی سه بعدی JavaScript است که برای اولین بار در Action Script توسعه یافت و سپس در سال ۲۰۰۹، به JavaScript منتقل شد. در سال ۲۰۱۰ توسط ریکاردو کابلو (مستر دووب)^۱ در GitHub منتشر شد.

لازم به ذکر است که برای استفاده از کتابخانه‌ی Three.js، باید تا حد زیادی به زبان JavaScript مسلط باشید.

۱-۲. ویژگی‌ها

این کتابخانه، همانند هر کتابخانه‌ی دیگری دارای مزایا و معایبی است که در ادامه به شرح آن‌ها می‌پردازیم؛

۱-۲-۱. مزایا:

- یادگیری آسان
- مثال‌های آموزشی
- مستندات خوب
- کارایی بالا
- پشتیبانی از فرمت‌های مدلی فراوان
- داشتن ویرایشگر اختصاصی
- داشتن تمام قابلیت‌های WebGL با پنهان سازی پیچیدگی‌های موجود
- تطبیق با سایر کتابخانه‌های JavaScript
- فراوانی امکانات گرافیکی؛ مانند، پس‌پردازش برای انعکاس جلوه‌های ویژه

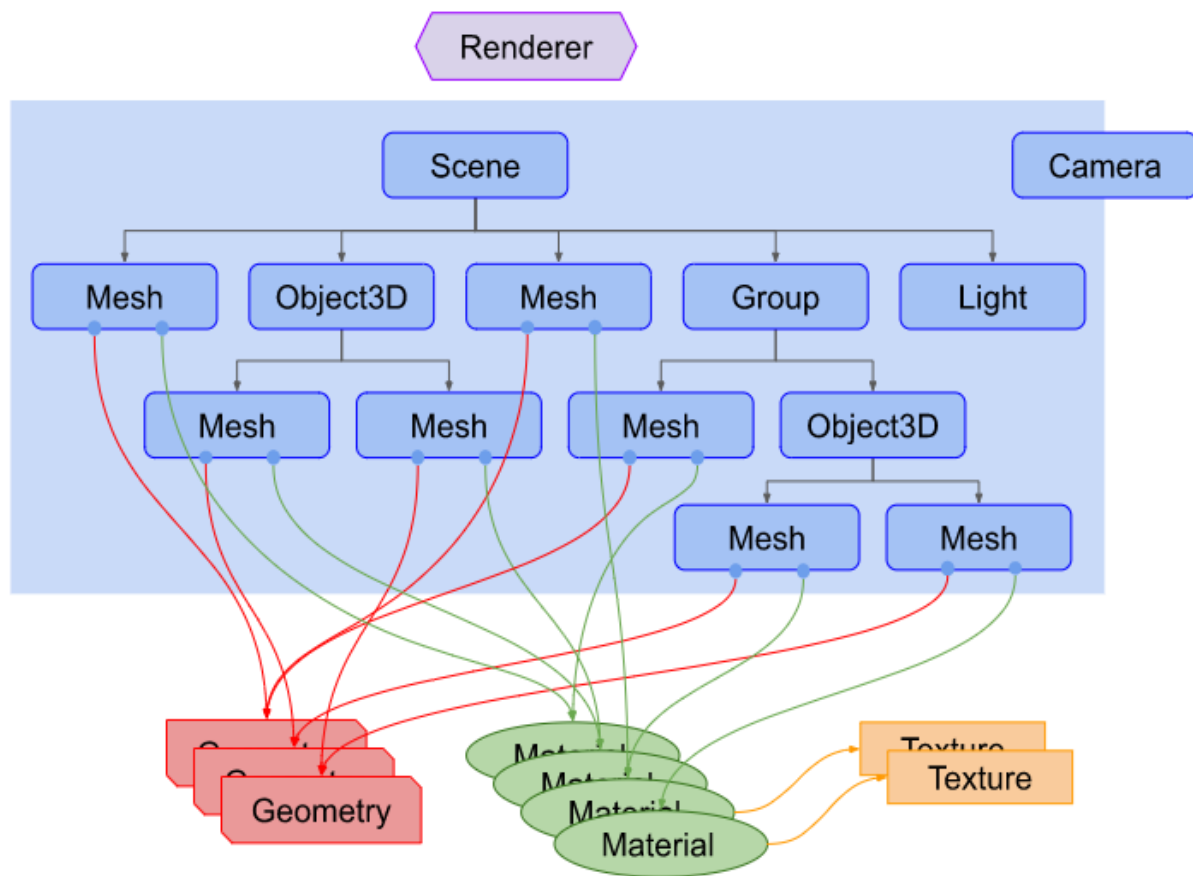
۱-۲-۲. معایب:

- دسترسی محدود به مستندات و منابع آموزشی به دلیل جدید بودن
- غیر قابل اجرا بودن بسیاری از تکنیک‌های مدرن
- موتور بازی نمی‌باشد.
- کارایی گرافیکی آن به مرورگرها و قدرت دستگاه‌ها وابستگی دارد.

- فاقد نسخه‌بندی بوده و دائما در حال بروزرسانی است.

یک برنامه‌ی Three.js شما را ملزم به ایجاد یک دسته از اشیا و اتصال آنها به یکدیگر می‌کند.

نمودار زیر نشان‌دهنده‌ی یک برنامه کوچک Three.js است.



شکل ۱-۱

۳-۱. معرفی پروژه:

هدف از این پروژه، پیاده‌سازی یک بازی تحت وب با استفاده از کتابخانه‌ی Three.js و قابل اجرای روی مرورگرهای وب است. در این بازی یک ماشین آمبولانس قصد دارد مسیری را در یک شهر طی کند. در این بین با ویروس‌های کرونا مواجه می‌شود که در واقع حکم موانعی را دارد که در صورت برخورد با آن‌ها، بخشی از انرژی خود را از دست می‌دهد. اگر به سمت این ویروس‌ها الکل بپاشد، امتیاز دریافت می‌کند و ویروس‌ها حذف می‌شوند. بازی اصطلاحاً بدون پایان^۱ می‌باشد؛ یعنی بردی ندارد، زیرا تعداد ویروس‌ها آنقدر زیاد است که کاربر به طور مداوم انرژی خود را از دست می‌دهد و در نهایت می‌بازد. اما با توجه به مهارت کاربر، این باخت ممکن است زودتر یا دیرتر (به‌وسیله‌ی از بین بردن ویروس‌ها با الکل پاشی) اتفاق بیفتد.

فصل دوم

معرفی نرم افزار

۲-۱. Visual Studio Code

ویژوال استودیو کد (یا به اختصار VSCode) کد در تاریخ ۲۹ آوریل سال ۲۰۱۵ توسط مایکروسافت در کنفرانس بیلد معرفی شد و پس از مدت کوتاهی یک نسخه پیش نمایش از آن معرفی شد. در ۱۸ نوامبر همان سال ویژوال استودیو کد، تحت پروانه ام‌آی‌تی در سایت Github.com منتشر شد.

در نظرسنجی سال ۲۰۱۸ وب سایت stackoverflow.com، ویژوال استودیو کد به عنوان محبوب‌ترین ابزار توسعه با رای ۳۴,۹ درصد از ۷۵,۳۹۸ رای انتخاب شد.

۲-۱-۱. کاربردها و ویژگی‌ها:

از کاربردهایی که این نرم‌افزار دارد می‌توان به موارد زیر اشاره کرد:

- کدنویسی و پشتیبانی گسترده از تگ‌ها و زبان‌های برنامه‌نویسی مختلف از جمله HTML، PHP، Python، JAVA، JavaScript، CSS، C++، C#، Visual Basic، SQL، XML و ...
- ویرایشگر کد متن باز برای سیستم عامل‌های لینوکس، مک و ویندوز

و...

همچنین دارای ویژگی‌های زیر است:

- نصب سریع و آسان
- کم حجم، در عین حال قدرتمند و پرسرعت

- پشتیبانی درونی از تکمیل کد هوشمند^۱، برجسته‌سازی نحو^۲، بازسازی کد^۳، عیب‌یابی^۴ و تکه کدها^۵

۲-۱-۲. قابلیت‌ها و مزایا:

برخی از قابلیت‌های این ویرایشگر عبارتند از:

- نصب تعداد زیادی افزونه (extention) برای زبان‌های مختلف
- ویرایش و اشکال‌زدایی دو فایل به‌صورت هم‌زمان در کنار هم برای مقایسه
- دارا بودن نقشه کد
- تکمیل خودکار کدها
- جستجوگر هوشمند و امکان پرسش مستقیم به توابع و متدها
- ترمینال دستورات متنی
- پشتیبانی از فناوری‌های Git
- امکان استفاده از Source control

1.intelligent code completion

2.syntax highlighting

3.code refactoring

4.debugging

5.snippets

- اشکال زدایی^۱ سریع و آسان کدها
- پوسته‌های رنگی و تیره محیط ویرایشگر

این نرم‌افزار مزایای زیادی دارد که به طور خلاصه چند نمونه از آن‌ها را ذکر می‌کنیم:

- دسترسی آسان و رایگان
- کاربر پسند بودن
- به‌روزرسانی سریع
- ذخیره‌ی جزئیات تمام ویرایش‌ها و تغییراتی که کاربر در پروژه ایجاد می‌کند.

قطعا این نرم‌افزار ویژگی‌ها، قابلیت و مزایای بیشتری دارد که در صورت استفاده از آن متوجه‌ی آن‌ها خواهید شد.

1.debugging

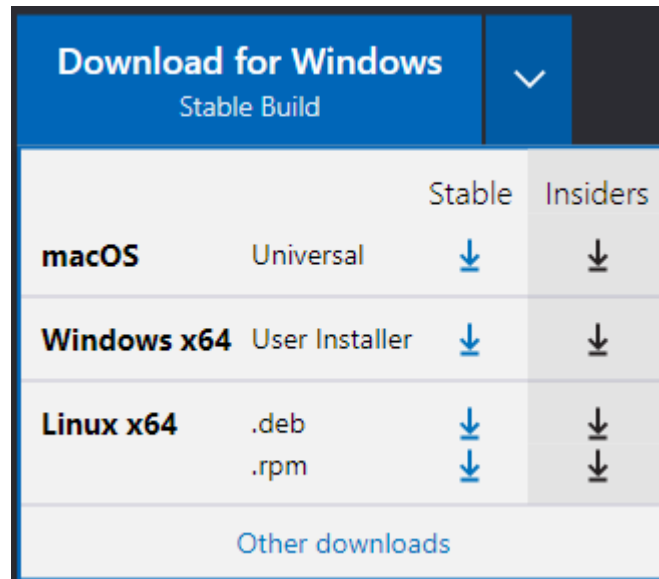
2.Theme

فصل سوم

نحوه‌ی کار با نرم‌افزار

۳-۱. دانلود نرم افزار

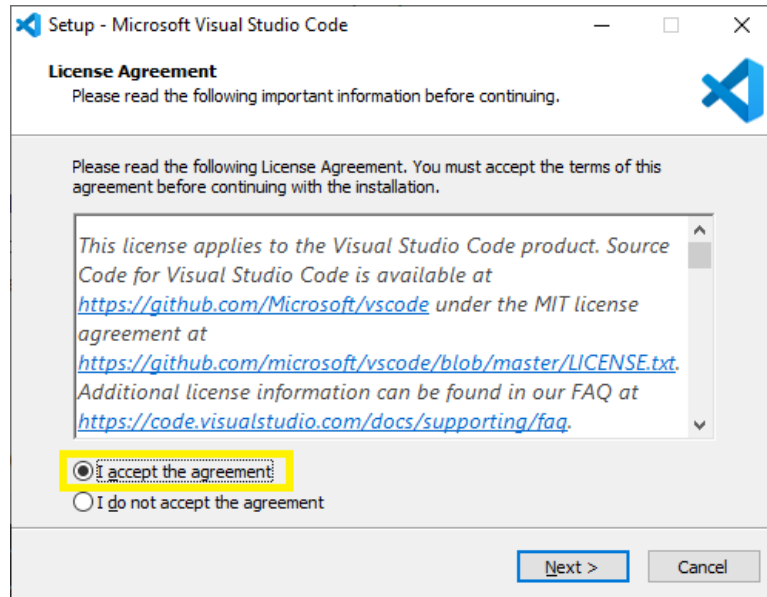
برای این کار کافی است به سایت <https://code.visualstudio.com/> مراجعه کرده و از گزینه‌ی Download for Windows سیستم عامل مورد نظر خود را انتخاب کرده و فایل .exe را دانلود کنید.



شکل ۳-۱

۳-۲. نصب

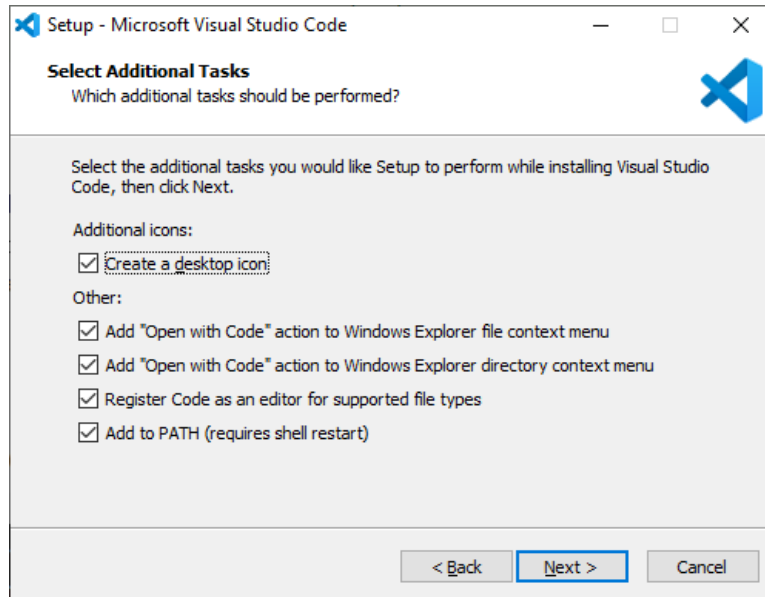
برای نصب هم کافی است فایل دانلود را باز کرده و همانند نصب سایر نرم افزارهای کامپیوتری، ابتدا با انتخاب I accept the agreement و سپس دکمه‌ی Next توافق نامه را بپذیرید.



شکل ۳-۲

سپس ادامه‌ی مراحل را با زدن دکمه‌ی Next رد کنید. (پیشنهاد می‌شود محل نصب پیش فرض نرم افزار را تغییر ندهید).

به این مرحله که در شکل ۳-۳ مشاهده می‌کنید، می‌رسید که پیشنهاد می‌شود تمامی موارد را تیک بزنید.



شکل ۳-۳

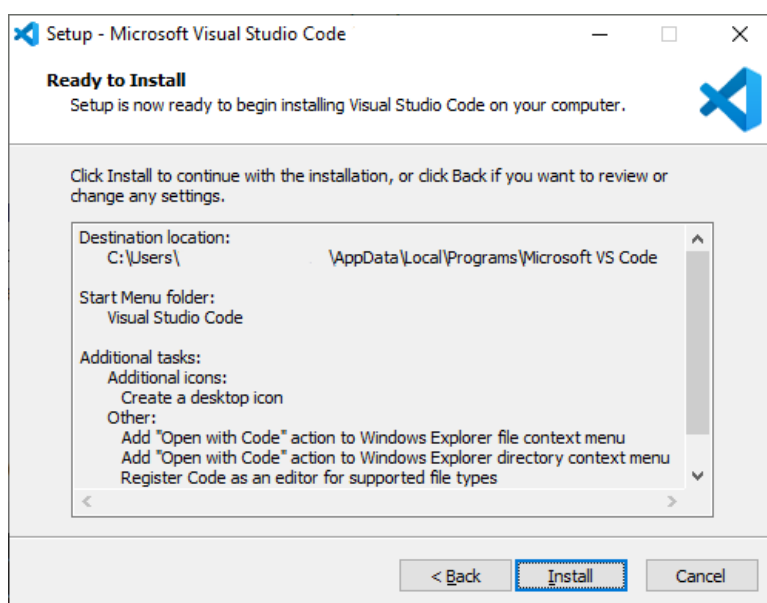
- گزینه Create a desktop icon بر روی صفحه دسکتاپ شما یک میانبر می‌سازد.
- گزینه Add "Open with Code" action to Windows Explorer file context menu به شما این امکان را می‌دهد تا با انتخاب فایل و کلیک راست، گزینه Open with code برای ویرایش هر فایلی نمایان شود.
- گزینه Add "Open with Code" action to Windows Explorer directory context menu به شما این امکان را می‌دهد تا با انتخاب پوشه و کلیک راست، گزینه Open with code برای افزودن پوشه به محیط کار (Workspace) نمایان شود.
- گزینه Register Code as an editor for supported file types تمام فایل‌های متنی با پسوند پشتیبانی شده در سیستم شما با VS Code باز می‌شوند. (مانند .cpp و .py)

- گزینه Add to PATH (requires shell restart) عبارت code را به آدرس PATH ویندوز اضافه

می کند که به شما امکان اجرای VS Code با دستور code از خط فرمان را می دهد. این گزینه را

حتما فعال کنید.

حال برنامه آماده نصب می باشد. Install را انتخاب کنید.

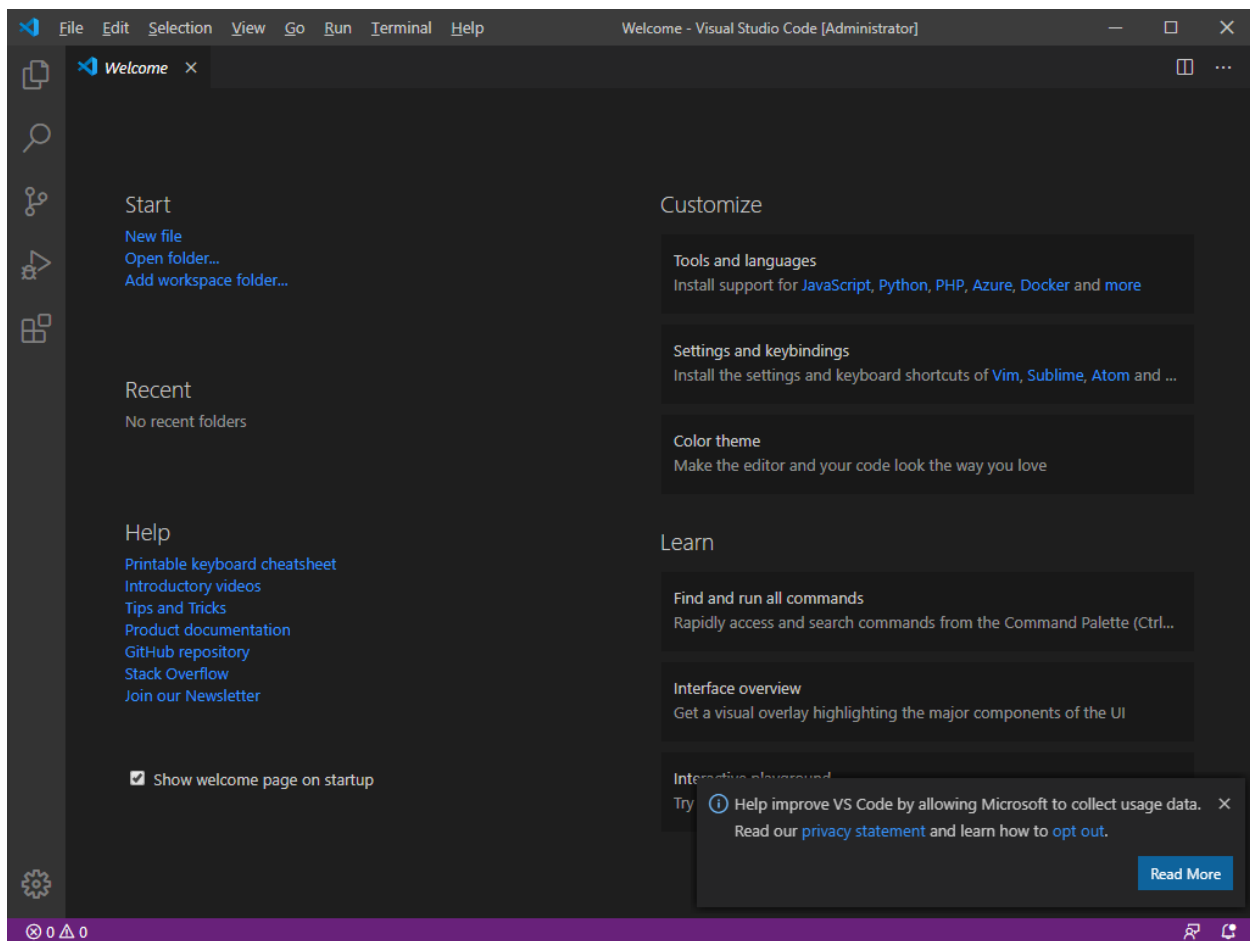


شکل ۳-۴

صبر کنید تا نصب تمام شود. برنامه VS Code با موفقیت بر روی سیستم شما نصب شده است. با انتخاب

Launch Visual Studio Code و سپس Finish آن را اجرا کنید.

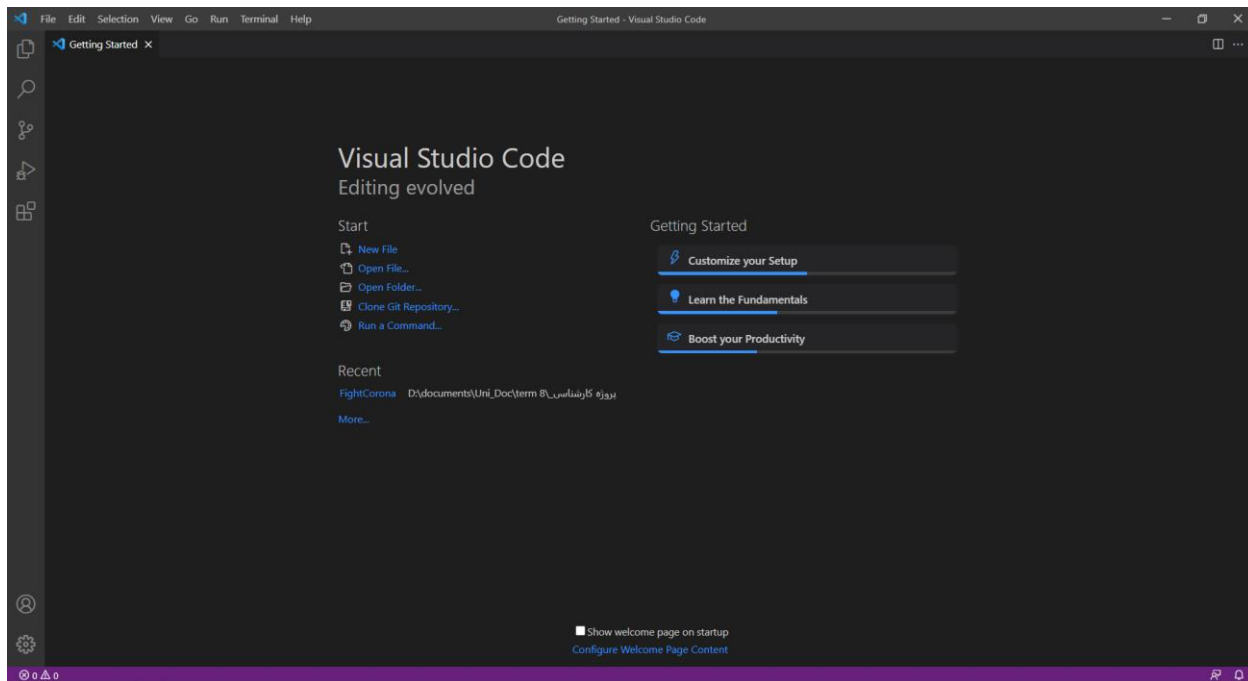
صفحه ای همانند شکل ۳-۵ برای شما باز می شود.



شكل ٣-٥

۳-۳. ایجاد فایل

با باز کردن Visual Studio Code، در صفحه Welcome در قسمت بالا سمت چپ و بخش Start، با زدن گزینه New File، یک فایل جدید ایجاد و نشانگر ماوس در ابتدای آن فایل قرار می‌گیرد.



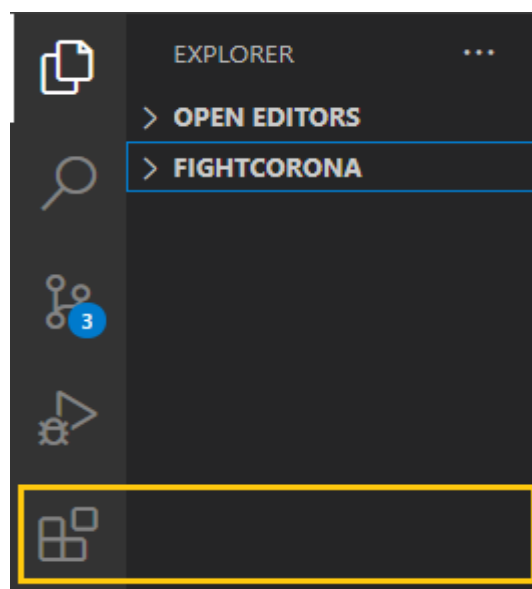
شکل ۳-۶

فایل مورد نظر را نوشته و با پسوند مناسب ذخیره می‌کنید.

۳-۴. نمایش خروجی

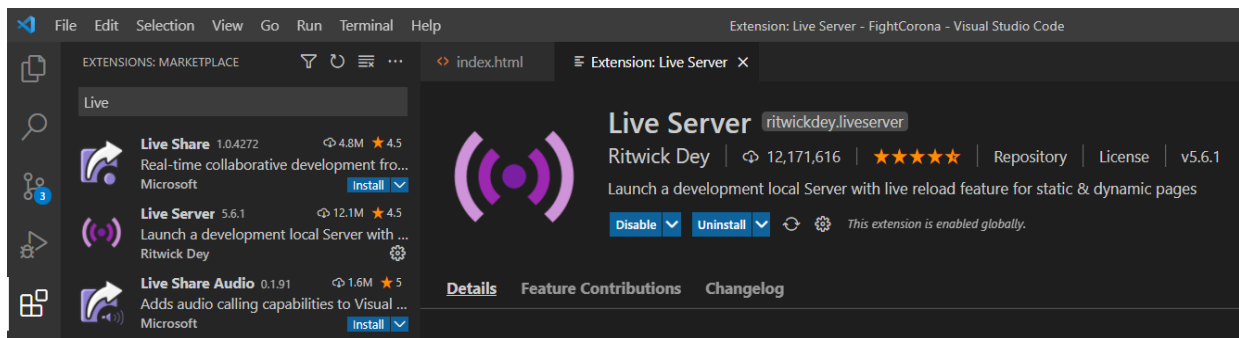
برای آن که بتوانید در برخورد با فایل‌هایی همانند این پروژه (HTML) راحت‌تر خروجی بگیرید، کافی است افزونه‌ی Live Server را نصب کنید.

برای نصب افزونه باید ابتدا به قسمت Extentions در سمت چپ نرم‌افزار مراجعه کنید.



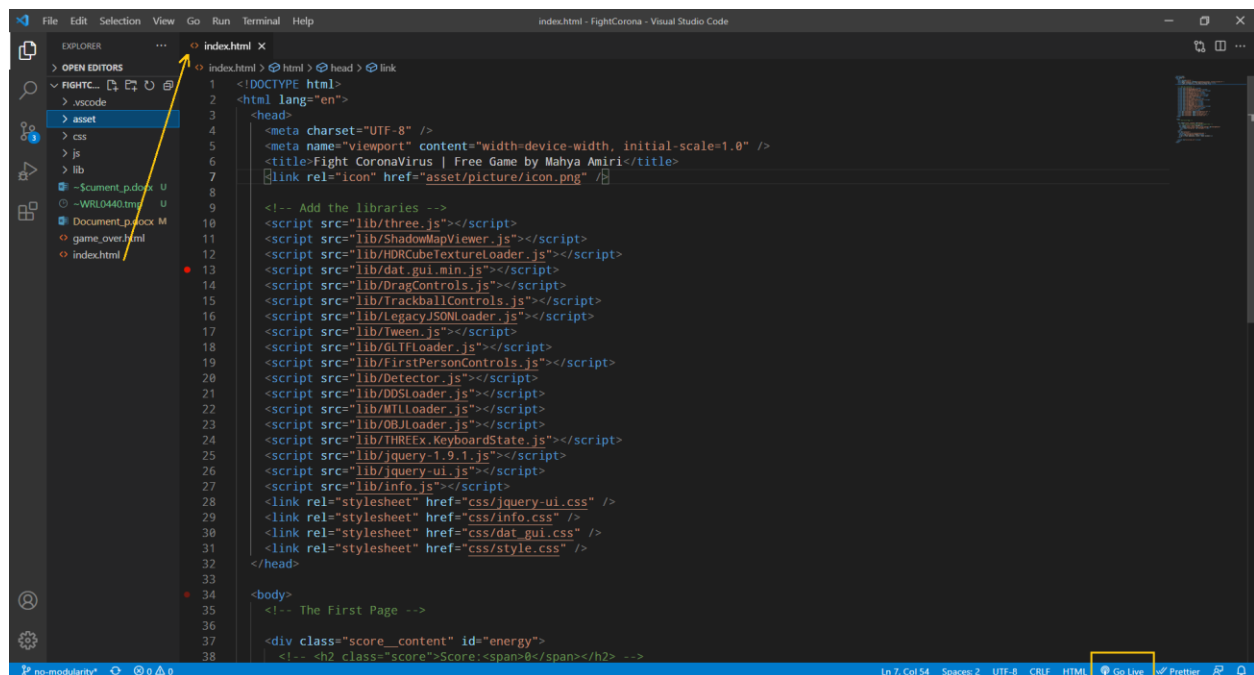
شکل ۳-۷

حال اسم افزونه را جستجو می‌کنیم و آن را نصب می‌کنیم.



شکل ۳-۸

اکنون فایل `index.html` مورد نظر را در ویرایشگر باز کرده و برای نمایش خروجی بر روی `Go Live` که در قسمت پایین سمت راست قرار دارد، کلیک می‌کنیم.



شکل ۳-۹

فصل چهارم

طراحی

در این فصل ارکان اصلی برای طراحی یک بازی به کمک این کتابخانه را ذکر می‌کنیم و درمورد نحوه‌ی عملکرد و ضرورت هر کدام توضیح می‌دهیم.

۱-۴. کتابخانه و ساختار کلی برنامه

پیش از نوشتن کدهای مورد نظر، کتابخانه‌های لازم را به فایل HTML اضافه کنید.

برای این کار می‌توانید به آدرس <https://github.com/mrdoob/three.js/tree/master/src> مراجعه کرده و یا از طریق لینک این کتابخانه‌ها اضافه کرد و یا برای آن‌ها در پوشه مورد نظرتان، فایل‌هایی ایجاد کنید و آن فایل‌ها را آدرس دهی کنید. این کتابخانه‌ها حجم زیادی اشغال نمی‌کنند، پس لازم نیست نگران حجم پوشه‌ی پروژه‌ی خود باشید.

پیشنهاد می‌شود از اضافه کردن کتابخانه‌هایی که برای پروژه‌ی شما کاربرد ندارند، خودداری کنید. در این صورت کدهای شما تمیزتر خواهد بود و پروژه گیج‌کننده نمی‌شود.

همان‌طور که مشخص است، اصلی‌ترین کتابخانه Three.js می‌باشد.

ساختار کلی برنامه بدین صورت خواهد بود:

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My first three.js app</title>
    <style>
      body { margin: 0; }
    </style>
  </head>
  <body>
    <script src="js/three.js"></script>
    <script>
      // Our Javascript will go here.
    </script>
  </body>
</html>

```

شکل ۴-۱

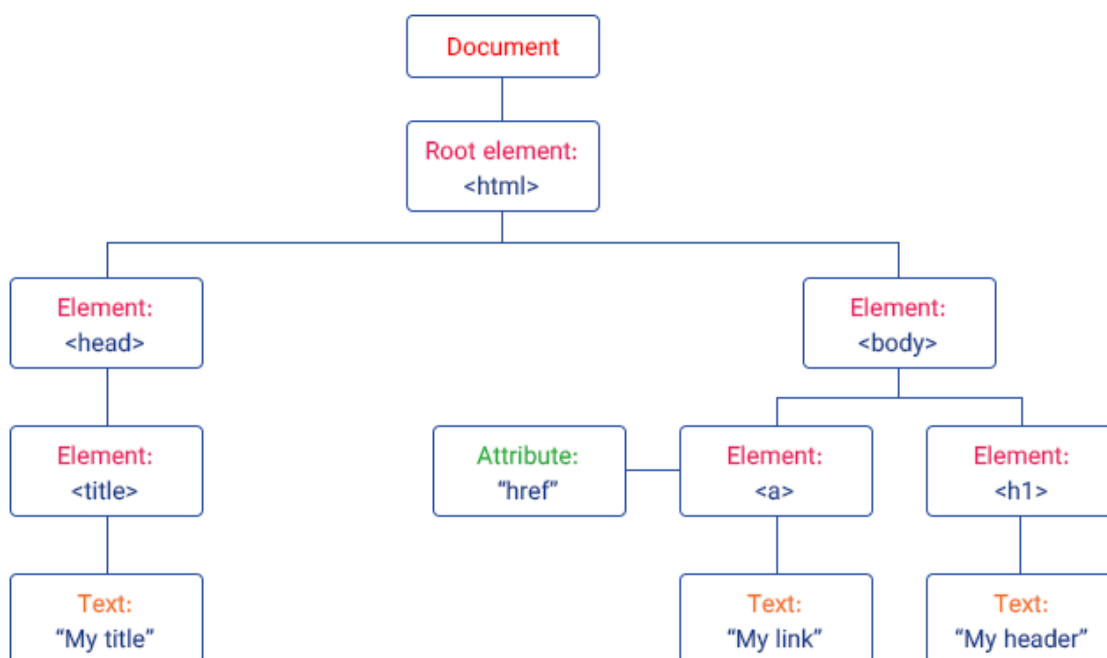
۴-۲. درخت DOM

زمانی که یک صفحه‌ی وب بارگذاری می‌شود، مرورگر از آن صفحه، یک مدل شیء گرا^۱ ایجاد می‌کند. DOM یک مدل و ساختار درختی از تمام عناصر HTML درون یک صفحه‌ی وب می‌باشد که در آن عناصر HTML به عنوان اشیاء JavaScript در نظر گرفته می‌شوند.

1.Document Object Model

این اشیاء، مانند سایر اشیاء JavaScript دارای تعدادی خاصیت و متد هستند. همچنین این اشیاء دارای یک ساختار سلسله مراتبی هستند. یعنی هر یک از عناصر صفحه می‌تواند تعدادی عنصر فرزند^۱ و یک عنصر والد^۲ داشته باشد.

در اسناد HTML تمام تگ‌ها داخل یک تگ اصلی به نام `<html>` تعریف می‌شوند. بنابراین ریشه‌ی این ساختار درختی همان تگ `<html>` خواهد بود که معمولا دو فرزند به نام‌های `<head>` و `<body>` دارد. در مدل DOM کل این ساختار درختی به عنوان فرزند شیء دیگری به نام `document` در نظر گرفته می‌شود.



شکل ۲-۴

1.Child

2.Parent

این موضوع در پروژه به این صورت استفاده شده است که کد اسکریپتی که ما درون تابع `init()` می‌نویسیم، به منظور شناسایی و اجرا حتماً باید به درخت DOM افزوده شود. بنابراین ما عنصر `renderer` را به سند HTML اضافه می‌کنیم. این یک عنصر `<canvas>` است که `render` برای نمایش صحنه به ما، استفاده می‌کند.

۳-۴. تابع `animate()`

لازمه‌ی اجرای یک برنامه بر روی وب و نمایش اجزای بازی، یک حلقه `render` یا `animating` می‌باشد. این حلقه باعث می‌شود `render` در هر بار `refresh` شدن صفحه، صحنه را مجدد ترسیم کند. (در یک صفحه‌ی معمولی این یعنی ۶۰ بار در ثانیه).

`requestAnimationFrame()` به مرورگر می‌فهماند که شما می‌خواهید یک انیمیشن (مثل بازی) را اجرا کنید و از مرورگر می‌خواهد که یک تابع مشخص را فراخوانی کند تا انیمیشن را قبل از رنگ‌آمیزی بعدی به‌روز کند. یک تابع به صورت `callback`، به عنوان آرگومان (در این پروژه همان تابع `animate`) می‌گیرد تا قبل از رنگ‌آمیزی مجدداً فراخوانی شود. چون می‌خواهیم فریم به فریم، این صحنه در رنگ‌آمیزی‌های بعدی هم به‌روز شوند، پس خود `requestAnimationFrame()` را در تابع `animate()` فراخوانی می‌کنیم.

اساساً هر چیزی که لازم باشد هنگام اجرای برنامه تغییر کند، باید از حلقه‌ی `animate` عبور کند. می‌توانید در این حلقه سایر توابع را هم فراخوانی کنید تا در نهایت با یک تابع `animate` که صدها خط کد است، مواجه نشوید.

۴-۴. ساخت صحنه‌ی بازی

برای نمایش هر پروژه‌ای با Three.js به سه مورد نیاز داریم؛ صحنه، دوربین و render. به عبارت بهتر ما باید صحنه را با دوربین ارائه دهیم.

۴-۴-۱. صحنه^۱

اگر هر شی‌ای مانند نور، سطح، مدل سه بعدی و... که ما در برنامه می‌سازیم به صحنه اضافه شود، در نمایش خروجی بر روی صفحه‌ی وب ظاهر می‌شوند.

۴-۴-۲. دوربین^۲

برای مدیریت دوربین، آنرا به صورت یک شی مجازی که دارای محل و جهتی در فضا است در نظر می‌گیریم. برای انجام هر تبدیل روی دوربین، تبدیل بازیگر^۳ دوربین را در نظر می‌گیریم و آنرا وارونه می‌کنیم. برای ترسیم دوربین باید چند ویژگی را رعایت کنیم:

- 1.Scene
- 2.Camera
- 3.Actor

۱. زاویه دید بر حسب درجه^۱ (FOV): کنترل میزان باز بودن لنز دوربین. در واقع FOV میزان صحنه‌ای است که هر لحظه بر روی صفحه نمایش دیده می‌شود و مقدار آن بر حسب درجه است.

۲. نسبت ظاهری^۲: نسبت عرض به ارتفاع canvas. اگر این مورد را رعایت نکنیم، تصویری بدشکل خواهیم داشت.

۳. صفحه نزدیک و دور^۳: به ترتیب نزدیک‌ترین و دورترین محل اشیاء به دوربین است که هنوز در آن نقطه اشیاء دیده می‌شوند. در واقع با تعیین این دو مقدار، اجسام دورتر یا نزدیکتر render نخواهند شد.

چندین دوربین مختلف در Three.js وجود دارد که به بررسی هر کدام می‌پردازیم ولی معمولاً از دوربین عمودی^۴ و دوربین پرسپکتیو^۵ استفاده می‌شود.

.

1.Field of View

2.Aspect

3. Near, Far

4.Orthographic Camera

5.Perspective Camera

۱-۲-۴. دوربین آرایه‌ای^۱

از دوربین آرایه‌ای می‌توان به منظور ارائه کارآمد صحنه با مجموعه‌ی از پیش تعریف شده دوربین استفاده کرد. این یک جنبه عملکرد مهم برای ارائه صحنه‌های VR است. نمونه‌ای از دوربین آرایه‌ای همیشه دارای آرایه‌ای از دوربین‌های فرعی است. تعریف ویژگی viewport برای هر دوربین فرعی، اجباری است. زیرا بخشی از view را که با این دوربین ارائه می‌شود، تعیین می‌کند.

۲-۲-۴. دوربین مکعبی^۲

۶ دوربین ایجاد می‌کند که به WebGLCubeRenderTarget ارائه می‌شود.

۳-۲-۴. دوربین عمودی

دوربین از مکانیزم orthographic استفاده می‌کند. در این مکانیزم، اندازه‌ی یک شی در تصویر render شده بدون در نظر گرفتن فاصله آن از دوربین، ثابت می‌ماند. این مورد می‌تواند برای ارائه‌ی صحنه‌های دو بعدی و عناصر UI مفید باشد. توسط این دوربین اجسام دور از دوربین یا اجسام نزدیک به دوربین، هم‌اندازه دیده می‌شوند.

1.ArrayCamera

2.Cube Camera

۴-۴-۲-۴. دوربین پرسپکتیو

دوربینی که از طرح چشم انداز استفاده می‌کند. طراحی این مکانیزم از رویت چشم انسان بهره گرفته است. این رایج‌ترین مکانیزمی است که برای ارائه یک صحنه سه بعدی استفاده می‌شود. در واقع در این مکانیزم اجسام دور از دوربین، کوچکتر و اجسام نزدیک به دوربین، بزرگتر دیده می‌شوند.

۴-۴-۲-۵. دوربین استریو^۱

مثل یک دوربین پرسپکتیو دوگانه است. برای جلوه‌های سه بعدی کاربرد دارد.

Render .۴-۴-۳

روی اشیاء درون صحنه، محاسباتی را انجام می‌دهد. سپس از مرورگر می‌خواهد آن‌ها را در قالب مشخصی (مثلا WebGL) نمایش دهد.

1.Stereo Camera

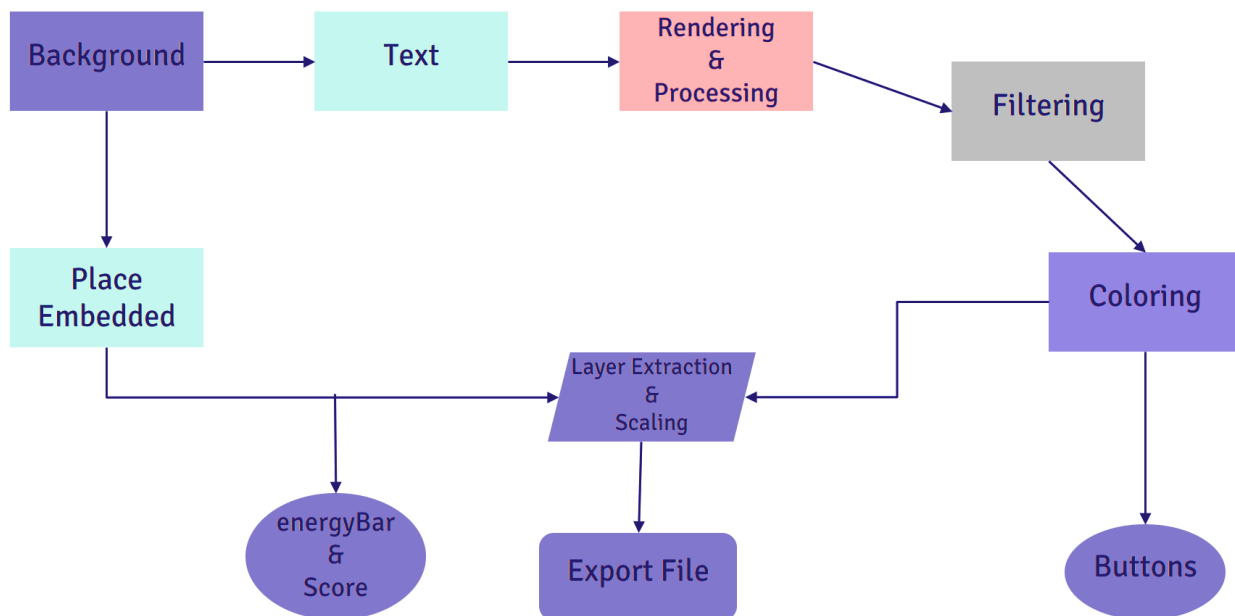
فصل پنجم

Front-end پروژه

در این فصل، front-end پروژه را به ترتیب بازی شرح می‌دهیم.

از آن‌جا که کرونا بحث تازه‌ای می‌باشد، هنوز تصاویر زیادی برای آن در اینترنت یافت نمی‌شود. بنابراین تصمیم گرفتم بخش front کار را خودم با کمک ابزارهای ویرایش تصویر (مثل photoshop) طراحی کنم.

برای روند طراحی یک فلوچارت در نظر گرفتیم:



شکل ۵-۱

برای هر طراحی‌ای فلوچارت از Background شروع می‌شود.

اگر بخواهیم صفحه‌ی شروع بازی و صفحه‌ی باخت را طراحی کنیم، از مسیر سمت راست در فلوچارت پیش می‌رویم. یعنی پس از انتخاب پس‌زمینه‌ی مربوطه، متن موردنظر را می‌نویسیم. سپس روی متن rendering و پردازش‌های مورد نظر را انجام می‌دهیم. فیلترگذاری می‌شوند. رنگ‌بندی متن را انجام می‌دهیم. سپس مقیاس-بندی نوشته‌ها را پس از استخراج لایه‌ها درست می‌کنیم و در نهایت فایل را export می‌کنیم.

همچنین دکمه‌هایی را که برای این دو صفحه لازم می‌باشد، ایجاد می‌کنیم.

اما اگر بخواهیم صفحه‌ی پس‌زمینه‌ی بازی را طراحی کنیم، باید مسیر سمت چپ را دنبال کنیم. کاری کمی ساده‌تر می‌شود زیرا با پردازش‌های متنی سر و کار نداریم. در این مسیر نیز پس‌زمینه‌ی مربوطه را انتخاب می‌کنیم. سپس تصاویر png را که قرار است در پس‌زمینه تعبیه شوند، اضافه می‌کنیم. مقیاس‌بندی و موقعیت مکانی آن‌ها را کنترل می‌کنیم و در نهایت فایل را export می‌کنیم.

انرژی و امتیاز هم در بخش front طراحی و در بخش back مدیریت می‌شوند.

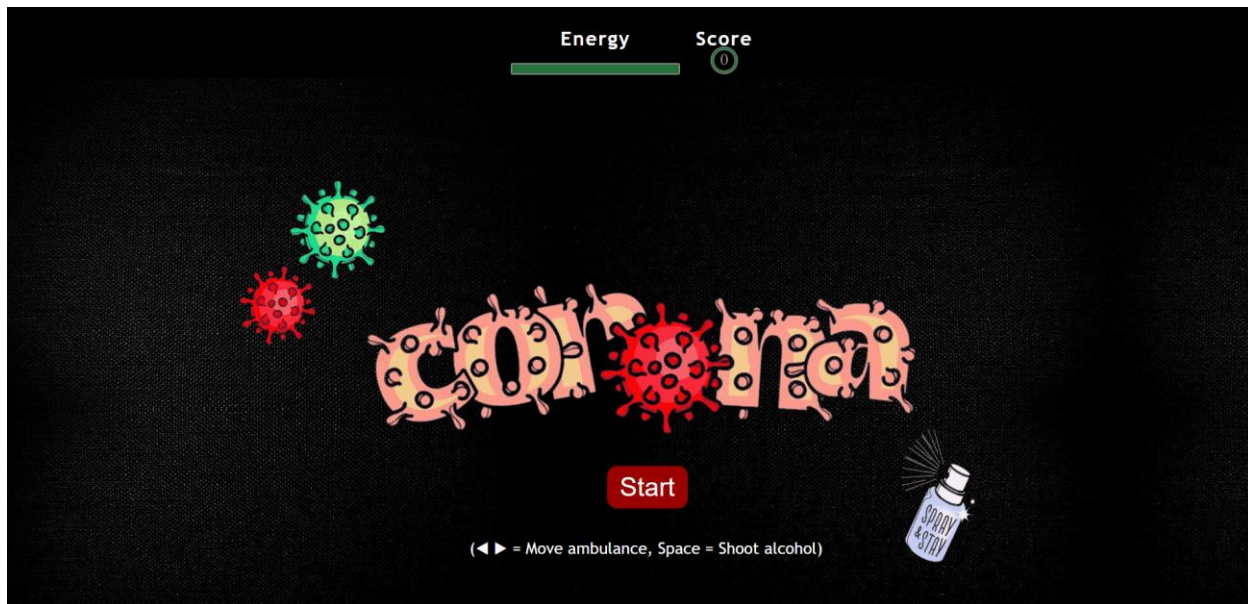
۱-۵. صفحه‌ی شروع بازی

طراحی این صفحه باید به گونه‌ای باشد که یک دکمه روی آن قرار داشته باشد که با زدن آن دکمه وارد بازی شویم. در مراحل اولیه، تصویری که برای این صفحه در نظر گرفته بودم، یک تصویر آماده از اینترنت بود.



شکل ۲-۵

اما با توجه به اینکه دکمه‌ی start در جای مناسبی قرار نمی‌گرفت و برای اینکه تم بازی را بهتر به کاربر القا کند، تصمیم گرفتیم خودم آن را به سلیقه‌ی خود و طبق اصول طراحی کنیم.



شکل ۳-۵

حتی اگر کاربری از قبل موضوع بازی را نداند، با دیدن همین صفحه هم متوجه می‌شود که موضوع بازی درباره‌ی ویروس کرونا می‌باشد. همانطور که در تصویر مشخص است، شیوه‌ی نوشتن واژه‌ی corona به گونه‌ای طراحی شده که خاصیت ویروس بودن آن را به کاربر می‌رساند. یک آیکون اسپری الکل در تصویر تعبیه شده که فلسفه‌ی الکل زدن را یادآور شود.

این صفحه شامل سه بخش است:

- دکمه‌ی Start
- Energy Bar
- امتیاز

۵-۱-۱. دکمه Start

برای وارد شدن به صفحه‌ی اصلی است که در فایل index.html با کمک تگ button آنرا تعریف کردیم. در فایل style.css به آن استایل لازم را دادیم و با قرار گرفتن موس روی دکمه، رنگ آن تغییر می‌کند. در فایل script.js پیش از تعریف تابع init() دکمه را با استفاده از id ای که در فایل index.html به آن داده بودیم، بازگردانیدیم و شرطی را تعیین کردیم که اگر بر روی دکمه کلیک شد، تابع init() فراخوانی و اجرا شود.

۵-۱-۲. Energy Bar

برای کنترل میزان انرژی از یک Energy Bar استفاده کردیم. برای این کار از تگ div استفاده کردیم و آنرا استایل‌دهی کردیم.

۵-۱-۳. امتیاز

برای کنترل مکانیزم پرتاپ الکل، یک بخش امتیاز در نظر گرفتیم. از تگ svg استفاده کردیم تا بتوانیم از گرافیک برداری مقیاس پذیر برای دایره‌ی قاب امتیاز کمک بگیریم و سپس استایل‌های لازم را به آن دادیم.

انرژی‌بار و قاب امتیاز دارای فونت نوشتاری خاصی هستند و در صفحه‌ی شروع بازی و صفحه‌ی اصلی برای کاربر نمایش داده می‌شود و در صفحه‌ی شروع به صورت حالت اولیه می‌باشد و تغییری نمی‌کند.

در فصل بعدی این دو قسمت را بیشتر توضیح خواهیم داد و به نحوه‌ی مدیریت آن‌ها خواهیم پرداخت.

۲-۵. صفحه‌ی پس‌زمینه‌ی بازی

پیدا کردن تصویر صفحه‌ی اصلی یکی از چالش‌هایی بود که در این پروژه با آن مواجه شدم. برای طراحی این صفحه حدود ۸ نسخه بازی ایجاد شد که ۳ مورد از نسخه‌های کامل‌تر آن‌ها موارد زیر بودند.

در ابتدای انجام پروژه تصویری که انتخاب کرده بودم به این صورت بود:



شکل ۴-۵

همانطور که پیداست شکل بازی بسیار نافرمان بود. زیرا texture جاده به خوبی روی جاده‌ی خود تصویر قرار نگرفته است و حتی بازیکن بازی، یک ماشین معمولی بود. بنابراین حتی اگر texture جاده به خوبی قرار می‌گرفت، با توجه به سناریوی بازی، شهر ما شرایط حاکم به دلیل وجود این ویروس را به مخاطب القا نمی‌کرد. بنابراین تصمیم گرفتم شهری را طراحی کنم که شرایط را به کاربر منتقل کند. (مثلا شهری خلوت که در آن مدافعان سلامت، آمبولانس و مردم عادی با ماسک در حال عبور می‌باشند. همچنین بازیکن یک آمبولانس باشد).

تصویر بعدی این گونه شد:



شکل ۵-۵

در این نسخه، شرایط سناریو بهتر به مخاطب منتقل می‌شود. اما بازهم مشکل اضافه شدن texture جاده و نافرمانی شدن تصویر وجود داشت.

در نهایت تصویری که طراحی کردم، به صورت زیر است:



شکل ۵-۶

در این طراحی یک تابلوی احتیاط به جاده اضافه شد و تصویری آمبولانسی که در شانه‌ی خاکی سمت چپ پارک شده هم عوض شد. همه چیز شکل‌تر و اصولی‌تر شد و همچنین texture جاده به خوبی روی جاده‌ی خود تصویر قرار می‌گرفت:



شکل ۵-۷

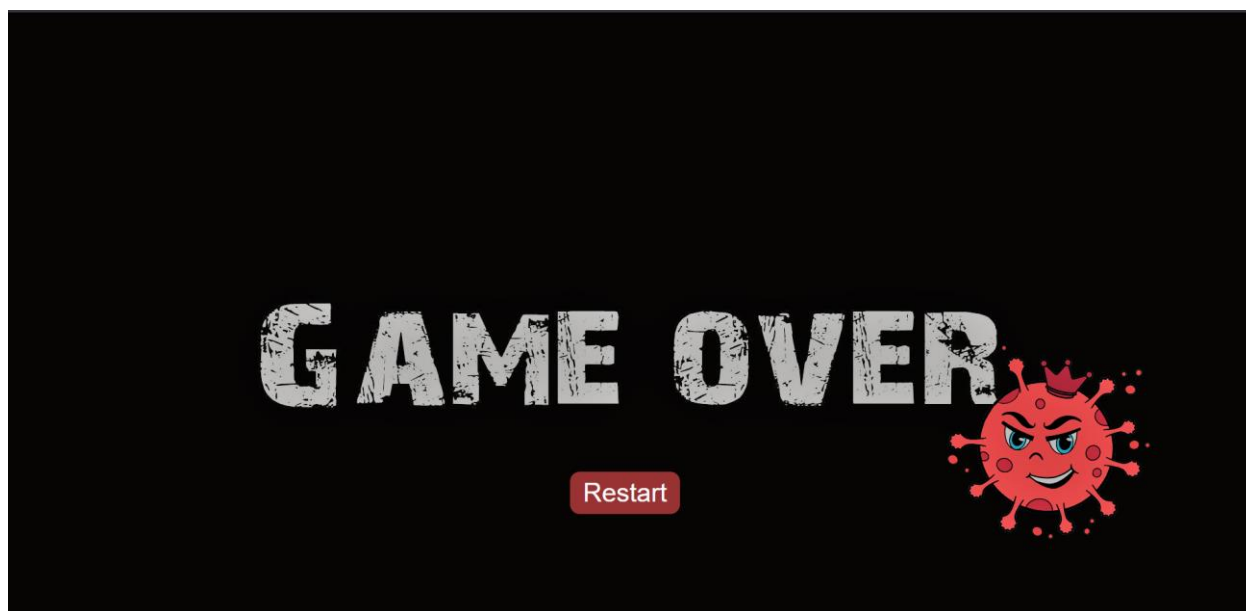
در این نسخه (که نسخه‌ی نهایی می‌باشد) مدل آمبولانس و شکل ویروس‌های کرونا را تغییر دادم تا جنبه‌ی fun بودن بازی را بیشتر کند. انرژی و امتیاز هم در بالای صفحه قرار گرفتند.

۵-۳. صفحه‌ی باخت بازی

همانند صفحه‌ی شروع بازی، یک صفحه برای باخت طراحی شده است. (چون بازی بدون پایان است، صفحه‌ی برد وجود ندارد).

برای طراحی این صفحه همانند صفحه‌ی شروع، متن را نوشته و پردازش‌ها و فیلترگذاری‌های لازم را انجام دادیم. آیکون ویروس کرونایی را اضافه کردیم که گویا از باخت کاربر خشنود است.

در فایل `game_over.html` کدهای لازم را اضافه می‌کنیم. خروجی این فایل بدین صورت است:



شکل ۵-۸

این قسمت شامل یک بخش است: دکمه‌ی Restart

۵-۳-۱. دکمه Restart

برای ورود مجدد به صفحه‌ی شروع طراحی شده است و مشابه طراحی دکمه‌ی Start می‌باشد.

همچنین یک صوت باخت هم برای این صفحه در نظر گرفته‌ایم.

فصل ششم

Back-end پروژه

۶-۱. مدیریت انرژی

طراحی انرژی بار به گونه‌ای است که انرژی کامل اولیه، با رنگ سبز برای کاربر نمایش داده می‌شود. در هر برخورد آمبولانس با کویدها، ۱۰٪ از انرژی کم می‌شود. اگر انرژی کمتر از نصف (۵۰٪) شود به رنگ قرمز درمی‌آید تا به کاربر این اخطار را بدهد که در آستانه‌ی باخت قرار گرفته است. همچنین اگر میزان انرژی کمتر از ۳۰٪ شود، انرژی به صورت چشمک زن در می‌آید و کاربر به باخت خیلی نزدیک شده و اگر انرژی کمتر از یک شود، کاربر باخته است.

از آن جا که بازی بدون پایان است، راهی وجود ندارد که انرژی زیاد شود، چون اصلاً قرار نیست در بازی بردی صورت بگیرد. اما خود کاربر می‌تواند انرژی را با پرتاب الکل مدیریت کند.

۶-۲. مدیریت امتیاز

هر بار که یکی از گلوله‌های الکل با یکی از ویروس‌ها برخورد کند، امتیاز یکی زیاد می‌شود و ویروس از صفحه‌ی بازی حذف می‌شود و با حذف ویروس باخت کاربر دیرتر صورت می‌گیرد.

برای افزایش امتیاز از متغیری کمک گرفته‌ایم که امتیاز اولیه (0) را از فایل html برگرداند و در هر بار برخورد یکی به آن اضافه کند. سپس به کمک تابع `math.floor()` بزرگترین عدد را بین امتیاز قبلی و عددی که به آن یکی اضافه شده، برگرداند.

۳-۶. سایه^۱

در قسمت render سایه را فعال کردیم تا در زمان ایجاد سایه برای هر شی، یک تصویر دو بعدی از آن شی روی اشیایی که سایه را دریافت می کنند، ایجاد شود.

باید در نظر داشت که برای ساخت سایه، حتما کتابخانه‌ی آن را در قسمت head تعریف کنید. (برای اطلاع از انواع سایه‌ها و ویژگی آن‌ها به بخش پیوست مراجعه شود).

۴-۶. سطح^۲

در این پروژه یک سطح تعریف شده که همان سطح جاده‌ی شهر می باشد. برای آن یک بافت^۳ به نام road.jpg بارگذاری کردیم. باید موقعیت مکانی آن را طبق تصویر بازی تنظیم می کنیم. این سطح می تواند سایه‌ی اشیاء دیگر را دریافت کند. آن را به صحنه اضافه می کنیم.

1.Shadow

2.Surface

3.Texture

۵-۶. بازیکن (آمبولانس)

بازیکن این بازی یک آمبولانس است. مدل سه بعدی آن را از <https://clara.io/> دانلود و در پروژه فایل json را با استفاده از متد ObjectLoader بارگذاری کردیم. باید توجه داشت که برای استفاده از این متد به کتابخانه خاصی نیاز داریم. مقیاس و موقعیت مکانی را تنظیم می‌کنیم. این شی هم می‌تواند سایه ایجاد کند و هم سایه دریافت کند.

مدل آمبولانس را به اندازه‌ی ۹۰ درجه در راستای محور y می‌چرخانیم. زیرا به طور پیش فرض سر آمبولانس به سمت راست بود و ما می‌خواستیم روبه جلو (در راستای منفی محور x) باشد.

آمبولانس را درون یک جعبه با خصیصه‌ی wireframe قرار می‌دهیم تا توسط این جعبه تشخیص برخورد آمبولانس با کویدها آسان‌تر شود. جعبه را نامرئی می‌کنیم تا در بازی دیده نشود. ابعاد جعبه‌ای که آمبولانس در آن است، برای تشخیص برخورد استفاده خواهد شد.

۶-۶. موانع

موانع در این بازی در واقع همان ویروس‌های کرونا هستند که در صورت برخورد با آنها انرژی کسر می‌شود.

الگوریتم ایجاد این ویروس‌ها به صورت شبه‌کد زیر است:

```
1: procedure createCovid()  
2: count ← 5000  
3: for each (index in count)
```



```

4:  {create covid
5:    position x for covid  $\leftarrow$  index  $\times$  200 - 10000
6:    position y for covid  $\leftarrow$  40
7:    position z for covid  $\leftarrow$  a random number  $\times$  800 - 450
8:    rotate covid in y direction by half PI
9:    set the setting for shadow and light
10:   position covid up
11:   add covid to the scene and arrays}

```

موقعیت z باید به گونه‌ای باشد که در هربار بازی این ویروس‌ها به صورت رندوم قرار بگیرند و موقعیتشان برای کاربر ثابت و قابل حدس نباشد. برای ایجاد یک ویروس، یک CircleGeometry در نظر می‌گیریم و عکس مورد نظر را به عنوان texture به آن اضافه می‌کنیم.



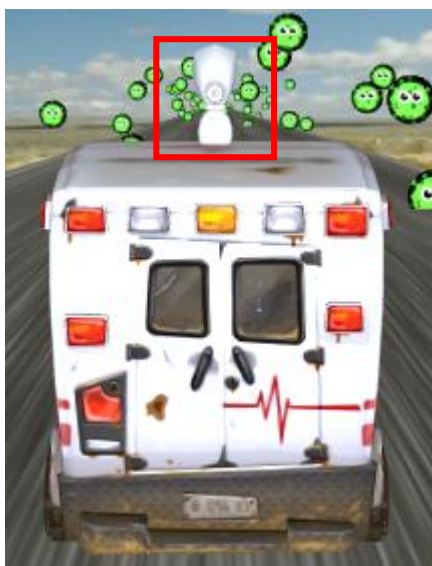
شکل ۶-۱

۶-۷. مکانیزم پرتاب الکل

برای نابود کردن ویروس‌ها باید به سمت آن‌ها گلوله‌های الکل را پرتاب کنیم. پس لازم است در بازی هم گلوله‌های الکل داشته باشیم و هم یک اسپری الکل‌پاش.

۶-۷-۱. Spray Gun

برروی سقف آمبولانس یک اسپری الکل‌پاش تعبیه شده است.



شکل ۶-۲

برای افزودن آن به صحنه‌ی بازی، همانند آمبولانس مدل سه بعدی‌اش را دانلود و فایل json. را اضافه کردیم.

Bullet ۶-۷-۲

برای هر گلوله‌ی الکل، یک کره با شعاع ۴ در نظر می‌گیریم و رنگ قرمز به آن می‌دهیم. از آنجا که این گلوله‌ها باید از اسپری شلیک شوند، موقعیت مکانی آن‌ها با توجه به موقعیت مکانی آمبولانس تنظیم می‌شود. برای جهت پرتاب گلوله‌ها از یک بردار سه بعدی استفاده می‌کنیم که با فاصله اقلیدوسی عمل می‌کند. سپس این گلوله‌های الکل اندکی پس از پرتاب، از صحنه حذف می‌شوند.

برای گلوله‌ها هم جعبه‌ای در نظر می‌گیریم (همانند آمبولانس) تا در صورت برخورد با وایروس بتوانیم این برخورد را تشخیص دهیم.

۶-۸. برخورد

برای برخورد باید رئوس جسم را در نظر بگیریم و بررسی کنیم اما با جعبه برخورد داشته یا نه. در واقع در این مدل برخورد، کاری با اضلاع نداریم.

در این بازی دو برخورد وجود دارد:

- برخورد آمبولانس با وایروس‌ها
- برخورد الکل با وایروس‌ها

۱-۸-۶. برخورد آمبولانس با ویروس‌ها

دو مختصات local و global را در نظر می‌گیریم. مختصات local یعنی مختصات خود جسم فارغ از این که در چه قسمتی از world قرار دارد. مختصات global همان چیزی است که کاربر می‌بیند یعنی در آن موقعیت دوربین و اثر پرسپکتیو هم دخیل است.

یک بردار جهت تعریف می‌کنیم که حاصل تفریق موقعیت مکانی جسم از مختصات global می‌باشد. عملکرد این بردار به این صورت است که نشان می‌دهد جسم در کدام جهت قرار گرفته و با حرکت دادن جسم برخوردی صورت می‌پذیرد یا نه.

با استفاده از مفهوم RayCast هر یک از موانع را به صورت جدا در نظر گرفتیم که در صورت برخورد آمبولانس با آن‌ها، انرژی کم می‌شود.

۲-۸-۶. برخورد الکل با ویروس‌ها

اگر هر گلوله‌ی الکل با یکی از ویروس‌ها برخورد کند، کاربر امتیاز می‌گیرد و آن ویروس حذف می‌شود. در این بخش هم مانند برخورد آمبولانس می‌باشد و تنها تفاوت آن در حذف ویروس می‌باشد.

به عنوان مثال، الگوریتم زیر مربوط به برخورد الکل و حذف کوید می‌باشد:

```
1: procedure movingCovid()  
2: time ← get elapsed time  
3: for each (index in objects)  
4:   {covid ← index  
5:     previousHeight ← position y for covid
```

```

6:      position y for covid ← ABS(SIN(index × offset + (time ×
speed) ÷ 2) × height)
7:      if (position y for covid < previousHeight) then
8:          (position covid down)
9:      else
10:         (position covid up) }

```

۹-۶. نور

در این بازی از دو نوع نور برای روشن کردن صحنه‌ی بازی و ایجاد سایه استفاده شده است: (برای اطلاع از انواع نورها و ویژگی آن‌ها به بخش پیوست مراجعه شود).

- نور محیط^۱
- نور جهت‌دار^۲

نور محیط، نوری است که به طور طبیعی (مثلا خورشید) یا مصنوعی (مثلا چراغ) وجود دارد. نوری ملایم است که صحنه‌ی بازی را پوشانده است؛ نه خیلی زیاد که باعث ناراحتی چشم شما شود و نه خیلی کم که هیچ اثری ایجاد نکند. نور جهت‌دار نوری است که در جهت خاصی به شیء تابیده می‌شود و تمرکزش روی آن نقطه بیشتر از سایر نقاط است. مثل ظهره‌نگام که نور خورشید به طور عمودی تابیده می‌شود.

1.Ambient Light

2.DirectionaI Light

۱۰-۶. صوت

یک موزیک بی کلام در پس زمینه‌ی بازی استفاده کردیم. دوربین را به عنوان یک شنونده در نظر گرفتیم. سپس موسیقی را play و آن را به صحنه اضافه کردیم. متد refDistance فاصله مرجع صوتی را برای کاهش میزان صدا نشان می‌دهد زیرا منبع صوتی از شنونده فاصله بیشتری می‌گیرد. یعنی باید فاصله‌ای که کاهش صدا شروع به کار می‌کند را به عنوان پارامتر به این متد بدهیم.

برای برخورد آمبولانس به وپروس‌ها و برخورد الکل به آن‌ها صوت خاصی را در نظر گرفتیم و آن را به روش دیگری تعریف کردیم.

۱۱-۶. حرکت

با فشار دادن کلیدهای راست و چپ صفحه کلید، آمبولانس و اسپری الکل به راست و چپ حرکت می‌کنند. از آنجا که یک جعبه برای تشخیص برخورد به آمبولانس اضافه کردیم، همراه جابه‌جایی آمبولانس خود جعبه هم به همان اندازه حرکت می‌کند.



شکل ۳-۶

همچنین اگر کاربر کلید Space صفحه کلید را فشار دهد، گلوله‌های الکل پرتاب می‌شوند.

۱۲-۶. دور شدن آمبولانس از دوربین

موقعیت مکانی X دوربین، آمبولانس و جعبه‌ی مربوطه، اسپری الکل‌پاش را به طبق فرمول زیر کم می‌کنیم:

$$x \text{ position} - 2 + \text{run}^{1.25} / 600$$

مقدار اولیه‌ی run صفر بوده و هر بار یکی به آن اضافه می‌شود.

این امر باعث می‌شود تا جاده روبه جلو حرکت کند. در دنیای واقعی نیز ما با این پدیده مواجه می‌شویم و این امر در واقع به دلیل خطای دید ما می‌باشد که احساس می‌کنیم جاده در حال حرکت است. در صورتی که جاده یک شی بی‌جان است و نمی‌تواند حرکت کند!

حرکت جاده در بازی باعث می‌شود که کاربر احساس کند که آمبولانس با سرعت ثابتی در حال حرکت است. در حالی که در دنیای واقعی ما اگر در یک نقطه بایستیم و حرکت یک اتومبیل را در نظر بگیریم، با دور شدن اتومبیل از ما اندازه‌اش هم در نظر ما کوچکتر می‌شود که دلیل آن دید پرسپکتیو چشم ما می‌باشد.

پس برای حرکت دوربین باید یک ضریب کاهشی در قسمت توان آن در نظر بگیریم که به خاطر این اختلاف کوچک، آهسته‌تر از آمبولانس و جعبه و اسپری حرکت کند. از آنجا که این تابع به صورت نمایی است، با گذشت زمان آمبولانس زودتر دور می‌شود. پس فرمول برای دوربین به صورت زیر خواهد بود:

$$x \text{ position} - 2 + \text{run}^{(1.25 - \text{reduction coefficient})} / 600$$

و از روش خطا و آزمون $\text{reduction coefficient} = 0.01$ بدست آوردیم.

۱۳-۶. تابع bounce()

این تابع برای بالا و پایین رفتن ویروس‌ها طراحی شده است.

برای این کار، ایده‌ی ابتدایی که در ذهن طراح جرقه می‌زند این است که جریان را طوری کنترل کنیم که وقتی کویدها به زمین برخورد کردند، در جهت معکوس یعنی رو به بالا حرکت کنند.

اما در این صورت با یک‌بار اجرا متوقف می‌شوند، چون تابعی که بوجود می‌آید متناوب نیست.

بنابراین اگر از تابعی استفاده کنیم که تناوب داشته باشد، مشکل برطرف می‌شود. پس باید حرکتشان را سینوسی بنویسیم.

در این تابع تابع تمامی کویدها به صورت سینوسی به سمت پایین حرکت می‌کنند و سپس موقعیت مکانی اولیه آن‌ها را درون یک متغیر ریخته و با موقعیت کنونی‌شان مقایسه می‌کنیم. اگر y کنونی آن‌ها کمتر از مقدار اولیه بود (یعنی از یک سینوسی به صفر سینوسی تغییر کرد) حرکت به سمت پایین بوده. پس با بررسی شرط "حرکت به سمت پایین است"، حرکت توپ این بار از پایین به بالا تغییر می‌کند.

فصل هفتم

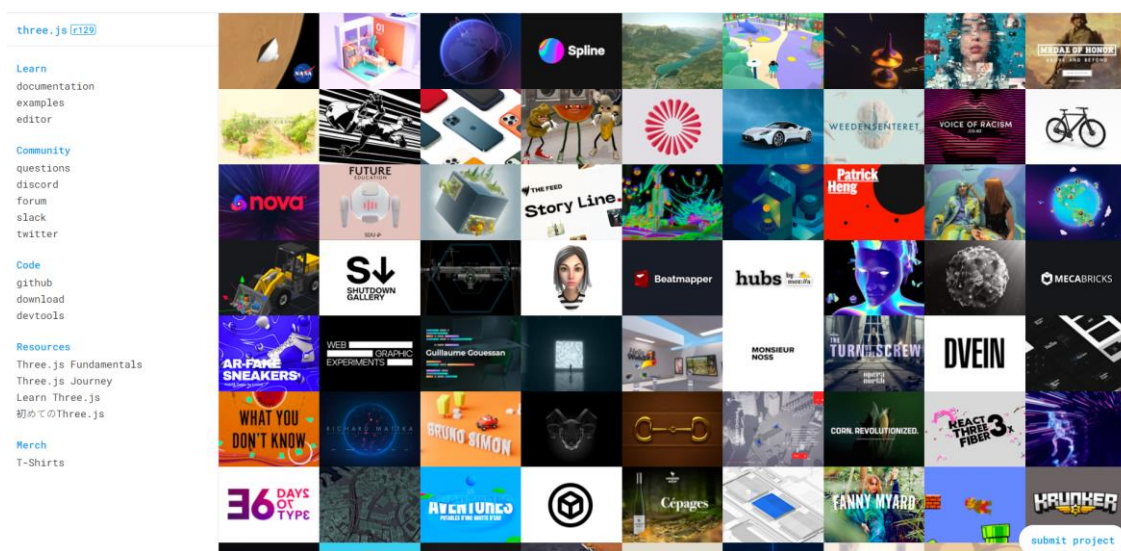
نتیجه گیری

۱-۷. نتیجه گیری

در دنیای امروزی بازی‌های کامپیوتری به گونه‌ای است که هر چه زمان بگذرد، بدون شک کیفیتش بهتر و خلاقانه‌تر خواهد بود و امکانات پیشرفته‌تری را در اختیار کاربر خواهد گذاشت.

کتابخانه‌ی Three.js بحث جدیدی است و جای پیشرفت بسیار زیادی خواهد داشت. برای نوشتن برنامه با آن فرد به دانش برنامه‌نویسی خیلی زیادی احتیاج ندارد و با دیدن مثال‌های مربوطه و تمرین و تکرار می‌تواند کارهای گرافیکی بسیار عالی و خلاقانه‌ای طراحی کند.

سایت اصلی <https://threejs.org/> به برنامه‌نویسان این امکان را می‌دهد تا از ابتدای ساخت صحنه تا ایجاد یک محیط گرافیکی پیشرفته، موارد لازم را بیاموزند و مطالب را با مثال‌های فراوان در اختیار آن‌ها قرار می‌دهد.



شکل ۷-۱

کدهای بسیاری از این مثال‌ها در سایت <https://github.com/mrdoob/three.js/tree/master> قرار دارد. می‌توانید از آن‌ها استفاده کنید.

در این پروژه ما با چالش‌های زیادی مواجه شدم. مثلاً یکی از آن‌ها تصویر مناسب برای پس‌زمینه بازی و قرارگیری texture جاده بود که توضیح دادم.

یکی دیگر از چالش‌ها، کاهش اندازه‌ی آمبولانس با گذشت زمان بود که توسط نوشتن فرمول و آزمون و خطا آن را برطرف کردم.

چالش دیگر که چالش بزرگی هم بود، مکانیزم پرتاب گلوله‌ی الکل بود که با دیدن ویدئوهای متعدد، بررسی مثال‌های زیاد و جستجو در گیت‌هاب موفق شدم آن چالش را البته با صرف زمان بسیار، صبر و شکیبایی حل کنم.

بسیار طبیعی است که هر پروژه‌ای که نوشته می‌شود، قابلیت تکمیل و بهبود دارد.

بنابراین در آینده قرار است زمان بیشتری صرف این بازی کنم تا بسیار حرفه‌ای‌تر شود و مواردی مثل particle system برای مکانیزم پرتاب الکل اضافه کنم تا به مفهوم اسپری نزدیک‌تر شود.

یا مثلاً در نظر دارم در نسخه‌های بعدی، به جای استفاده از یک تصویر پس‌زمینه، صحنه‌ی بازی را توسط اشکال هندسی ایجاد کنم و مواردی از هوش مصنوعی را در آن اضافه کنم.

منابع

لیست منابع

- [1]. <https://threejs.org/>
- [2]. <https://github.com/mrdoob/three.js/tree/master>
- [3]. <https://www.w3schools.com/js/>
- [4]. <https://clara.io/library>
- [5]. درسنامه‌های دکتر سید امیرهادی مینوفام

پیوست

پیوست ۱- انواع سایه و ویژگی‌های آن

به طور پیش فرض Three.js از shadow map استفاده می‌کند. روش کار shadow map به این صورت است که برای هر نوری که می‌تواند سایه ایجاد کند، همه اشیایی که برای ایجاد سایه علامت‌گذاری شده‌اند render می‌شوند. به عبارت دیگر، اگر شما ۲۰ شی و ۵ چراغ داشته باشید و همه‌ی آن‌ها سایه بیندازند، کل صحنه شما ۶ بار ترسیم می‌شود. هر ۲۰ شی برای نور شماره ۱ ترسیم می‌شوند، سپس برای نور شماره ۲ و... و در نهایت صحنه واقعی با استفاده از داده‌های ۵ رندر اول ترسیم می‌شود.

سایه‌ها دو نوع هستند:

- سایه سخت^۱
- سایه نرم^۲

سایه سخت

تیز و دارای رنگ یکنواخت، همچنین محاسبه آن‌ها سریع‌تر است ولی از واقع‌گرایی کمتری برخوردارند.

سایه نرم

ظاهر هموارتری دارند و به صورت تدریجی از مرکز به سمت لبه‌ها کم‌رنگ‌تر می‌شوند. واقع‌گرایانه‌تر ولی هزینه‌برتر هستند.

1.hard shadow

2.soft shadow

تکنیک‌های ساخت سایه

۱. تصویر کردن^۱: شکل تخت شده (دوبعدی) شی به عنوان سایه زیر خود شی قرار می‌گیرد.
۲. سایه حجمی^۲: دور جسم را بر اساس مکان/جهت منبع نور روی سطح رسم می‌کند و یک جسم جدید را به عنوان سایه می‌سازد.
۳. نگاشت سایه^۳: صحنه را از دید منبع نور می‌نگرد و با الگوریتم z-Buffer هرچه را نمی‌بیند سایه می‌زند.
۴. سایه نرم: چون محاسبه‌ی آن با روشنایی سراسری هزینه‌بر است، معمولاً به صورت جعلی یا تقریبی^۴ ایجاد می‌شود.

سایه نرم تقریبی با محاسبه‌ی دو بخش umbra و penumbra حاصل می‌شود؛

- Umbra: سایه کامل (تاریکی محض) است.
- Penumbra: سایه روشن (نیم سایه) است.

1.projection shadow

2.shadow volume

3.shadow mapping

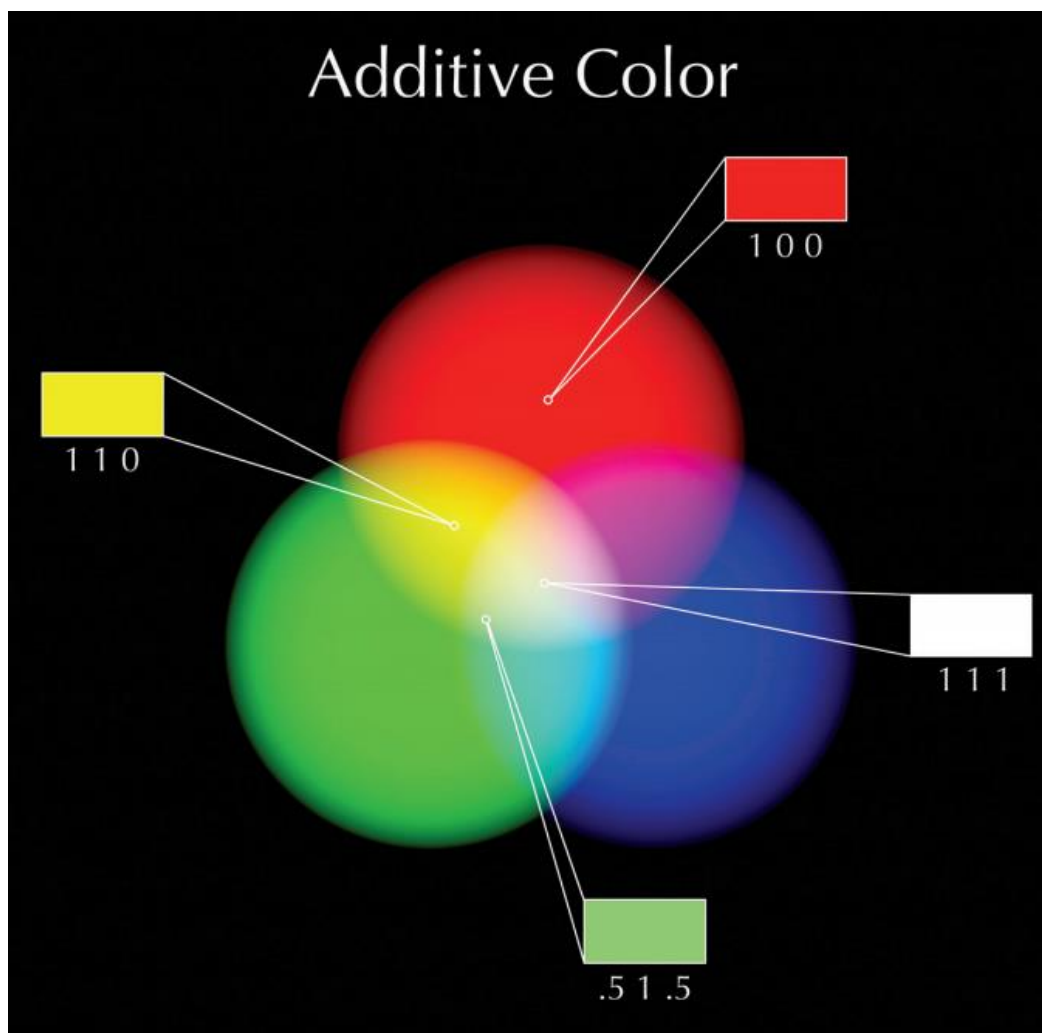
4.approximate

پیوست ۲- انواع نور و ویژگی‌های آن

رنگ نور

همه‌ی نورها در دنیای واقعی دارای رنگ هستند. حتی نورهایی که به دست انسان ساخته شده باشند (مثل لامپ‌ها) و این رنگ‌ها نیز از طیف گرم و یا سرد می‌باشند. نور خورشید، نور گرم از خودش تولید می‌کند. نورهای فلورسنت (مثل مهتابی) طیفی از رنگ سبز را ساطع می‌کنند. حتی نور تلویزیون از خودش رنگ‌های animate شده با طیف گسترده را ساطع می‌کنند. همه نورهایی که در بسته‌های نرم‌افزارهای مختلف سه‌بعدی در اختیارتان گذاشته شده است سفید است. پس به این نکته باید توجه داشته باشید که برای رندر گرفتن یک تصویر نهایی موفق و با کیفیت، تنظیم رنگ نور کاملاً ضروری و حیاتی است.

رنگ نورهای سه‌بعدی را می‌توانید از روش‌های گوناگون روی نورها اعمال کنید. اولین و برجسته‌ترین آن با استفاده از تکنیک RGB است. در این روش مقادیر RGB را به روش افزایشی (Additive) و با استفاده از مخلوطی از قرمز (R)، سبز (G) و آبی (B) ایجاد کنید. اگر تمامی مقادیر RGB را افزایش دهید، رنگ نوری سفید را خواهید داشت و اگر تمامی مقادیر را روی عدد صفر تنظیم کنید، رنگ نوری سیاه. اگر تمامی مقادیر را به یک اندازه قرار دهید، می‌توانید طیف وسیعی از مقادیر خاکستری را کنترل کنید. ولی اگر تمامی مقادیر عددی را متفاوت تنظیم کنید، می‌توانید رنگ بصری را که ترکیبی از RGB هستند را کنترل و از آن در رنگ نورهایتان استفاده کنید.



شکل پ. ۱

این اعداد می‌توانند از مقیاس ۰ تا ۱ و یا از ۰ تا ۲۵۵ تغییر کنند. مقادیر ۰ و ۱ که به سادگی می‌توانند درصد رنگ را مشخص کنند. به عنوان مثال اگر رنگ قرمز مد نظر شما باشد، مقدار R را ۱ قرار می‌دهید و اعداد G و B را روی صفر تنظیم می‌کنید، با اینکار شما رنگ قرمز را بدست خواهید آورد. مقیاس ۰ تا ۲۵۵ نیز به همان شیوه قابلیت اجرایی دارد اما نشان دهنده‌ی ارزش طیف وسیعی از یک تصویر ۸ بیتی نیست.

علاوه بر این، رنگ نورها را می‌توان به روش HSV (Hue, Saturation, Value) و یا HSL (Hue, Saturation, Lightness) به دست آورد. این متدها رنگ نور را بر اساس طیف Hue و شدت آن، و یا بر اساس

شدت روشن بودن آن مشخص و اعمال می‌کنند. مقدار Hue نیز که مقدار عددی از ۰ تا ۱ (یا گاهی اوقات بین ۰ تا ۳۶۰) را می‌گیرد. مقدار Hue را در چرخه رنگ کنترل می‌کند. اعداد بین ۰ تا ۱ معمولاً قرمز هستند و تمام اعداد بین این طیف رنگ‌های مختلف را مشخص می‌کند. مقدار Saturation نیز نشان دهنده‌ی میزان روشنی رنگ‌هاست و مقدار Value نیز نشان دهنده‌ی مقدار روشنی و یا تیرگی رنگ است.



شکل پ.۲

شدت نور

تنظیم کردن صحیح شدت نور نیز یکی از وظایف مهم و اصلی نورپردازها در انیمیشن است. نه تنها مناطقی از صحنه را روشنایی می‌بخشید، بلکه باید مقدار تاریکی صحنه را به طرز ماهرانه‌ای ایجاد کنید تا بتوانید چشم‌های بینندگان را به سوی سوژه (علاقه بصری) هدایت کنید.

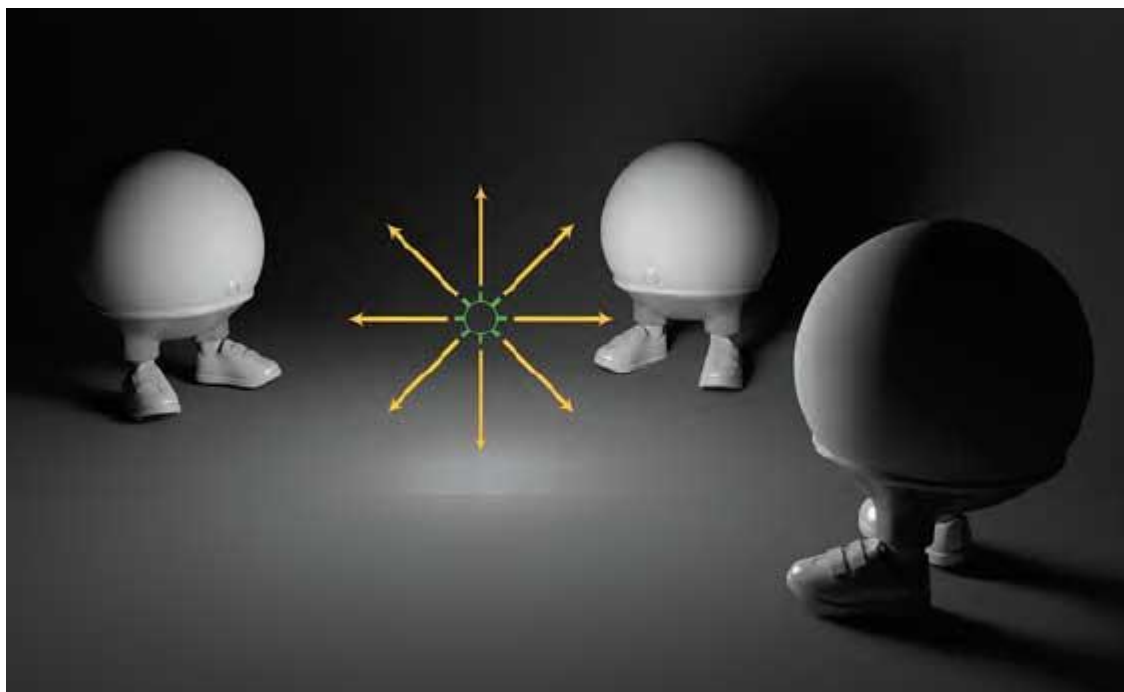


شکل پ.۳

انواع نور

نورهای نقطه‌ای

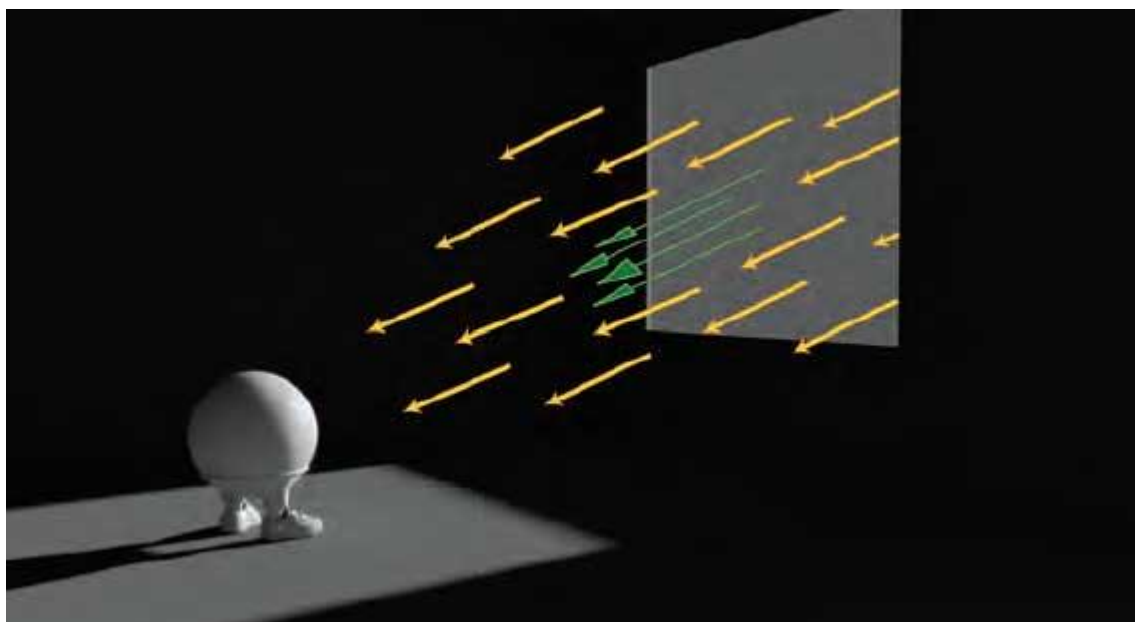
ساده‌ترین نوع هستند. با مختصات x و y و z معین در صحنه قرار می‌گیرند و در همه جهت از خود نور می‌تابانند. اگر سایه‌ها فعال باشند، آن‌ها هم از محلی که نور نقطه‌ای در آن قرار دارد، پخش می‌شوند. نورهای نقطه‌ای در شبیه سازی چیزهایی مثل نور یک شمع یا یک لامپ ساده، بهترین استفاده را دارند. اندازه‌ی نورهای نقطه‌ای کوچک است و همه جا را روشن می‌کنند. مخصوصاً زمانی کاربرد دارند که منبع نور بر روی صفحه‌ی نمایش باشد. در این حالت نور نقطه‌ای در حالی که به همه طرف نور می‌تاباند دیده می‌شود و سایه‌های حاصل از آن همانطور که در دنیای واقعی رفتار می‌کنند، پخش می‌شوند.



شکل پ.۴

نورهای جهت‌دار

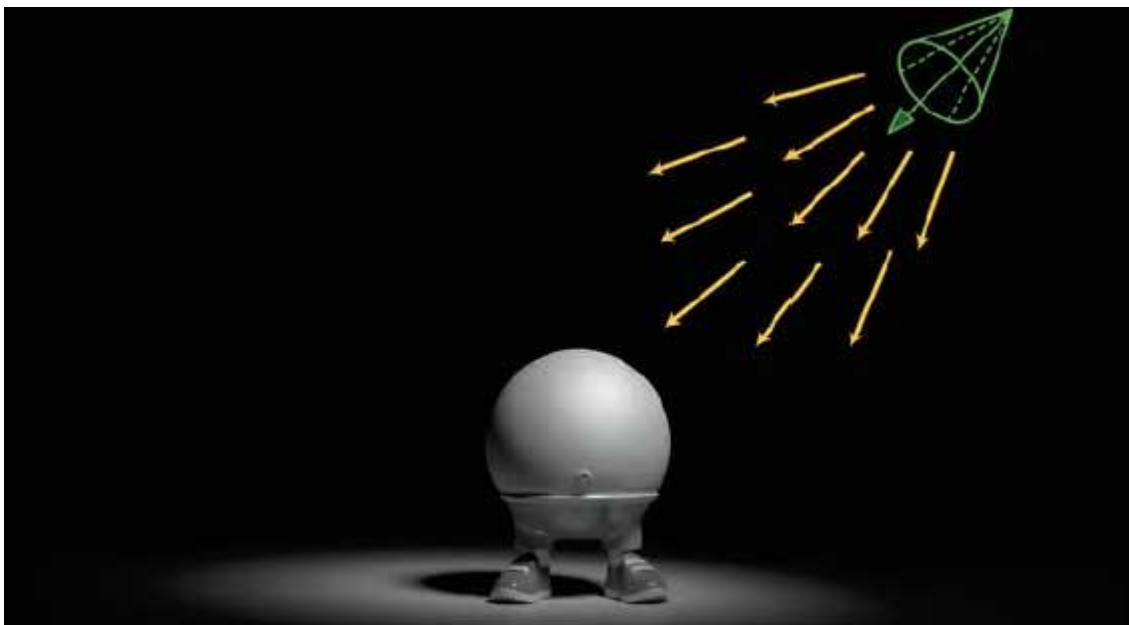
نورهای جهت‌دار یا نورهای فاصله‌دار، پرتوهای نور را در یک جهت مشخص و طبق زاویه‌ای که برنامه‌نویس معین کرده می‌تابانند. بنابراین در این مورد تنها ویژگی موضعی که اهمیت دارد، چرخش است. مختصات x, y, z و اندازه‌ی نور جهت‌دار مهم نیست. در نتیجه این نورها می‌توانند هر نوری که در فاصله‌ی بسیار دوری از صحنه قرار دارد را شبیه‌سازی کنند و تمام تصویر را روشن می‌کنند. به همین دلیل اکثراً ترجیح می‌هند برای شبیه‌سازی نور خورشید یا ماه، از نورهای جهت‌دار استفاده کنند.



شکل پ.۵

نورافکن

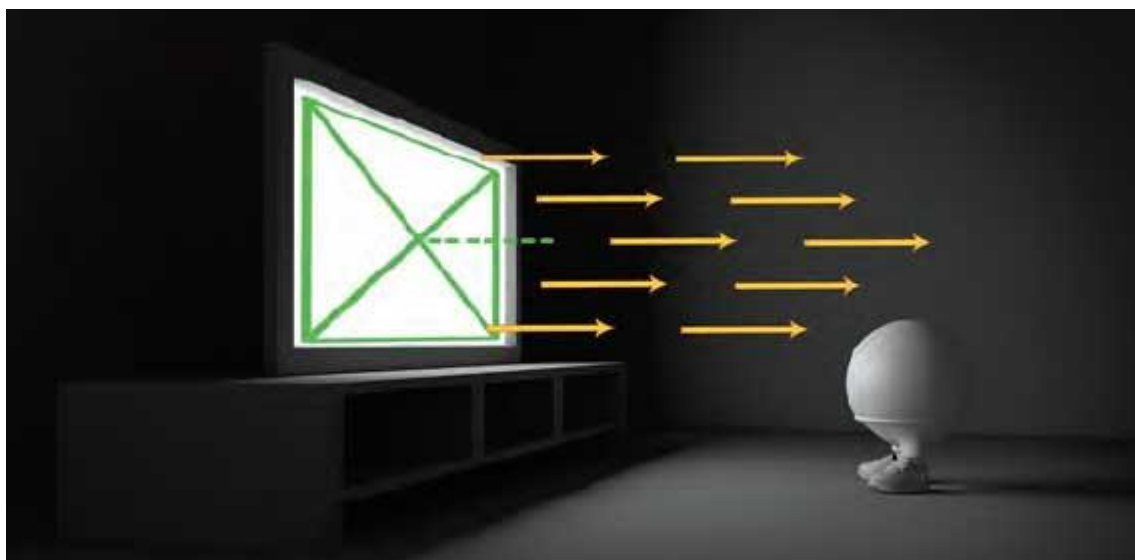
نورافکن‌ها در طیف گسترده‌ای کاربرد دارند. می‌توانند به عنوان چراغ قوه، چراغ خیابان، نورافکن صحنه نمایش و یا در هر جایی که نور از نقطه‌ای به شکل مخروطی می‌تابد، استفاده شوند. از این نظر که ابعاد فیزیکی‌ای ندارند به نور نقطه‌ای شبیه هستند. می‌تواند بر روی یک شیء خاص متمرکز شود. نورافکن‌ها بیشترین کنترل را بر روی تنظیمات سایه دارند که به هنرمند اجازه می‌دهد که بتواند هر نوع سایه‌ای را شبیه‌سازی کند. نورافکن‌ها برخلاف نور نقطه‌ای و نور جهت دار، پرتوهای نوری که تولید می‌کنند خیلی کمتر در خارج از صفحه به هدر می‌رود.



شکل پ.۶

نور سطحی

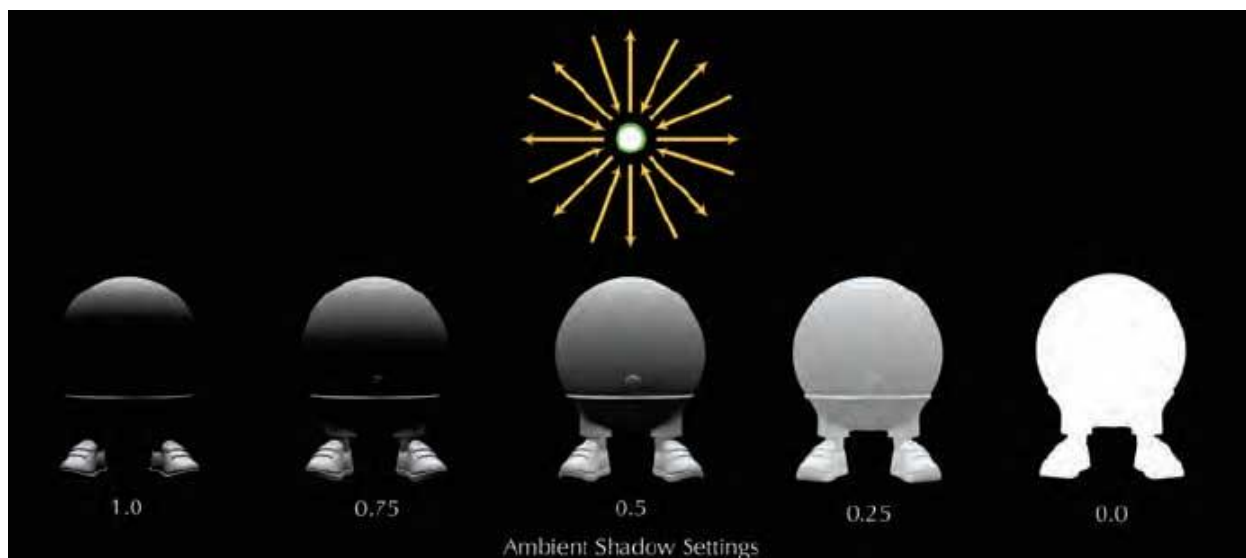
در واقع منبع نوری است که نور را از یک شکل هندسی می‌تاباند و در بسیاری از موارد بهترین گزینه برای نمایش منبع نور در دنیای واقعیست و معمولاً یک صفحه‌ی مسطح دو بعدی است. از آنجایی که نورهای سطحی از منبع نورهای نقطه‌ای و نورافکن‌ها بزرگتر هستند پس طبیعتاً نور ملایمی دارند. این نور سایه‌های ملایم و هایلایت‌های آینه‌ای بزرگی ایجاد می‌کند که ظاهر کلی لطیفی به تصویر می‌بخشد. معمولاً از نورهای سطحی برای نشان دادن مانیتورها، صفحه‌ی تلویزیون یا چراغهای بزرگ و مسطح فلورسنت استفاده می‌شود.



شکل پ. ۷

نور محیطی

در جهان واقعی وجود ندارد. به طور پیش فرض نور محیطی، تمام اشیاء موجود در صحنه را به صورت یکسان روشن می‌کند و هیچ سایه‌ای ایجاد نمی‌کند و جهتی ندارد. ملموس‌ترین مثال از نور محیطی در دنیای واقعی یک روز ابری است که ابرها به طور کامل جلوی نور خورشید را گرفته‌اند و همه چیز در محیط به طور یکنواخت روشن شده است.



شکل پ.۸